Atp Tennis Match Outcome Prediction Using Artificial Neural Networks

Kutay Koray

Purpose Of The Project

The goal

Develop a predictive model for ATP World Tour match outcomes with high accuracy, moving beyond linear statistical methods.

Complexity

Tennis performance is volatile, influenced by trends, momentum, surface type, and head-to-head history, making simple ranking systems insufficient.

Purpose Of The Project

From Scratch

To understand the concepts deeper.

Experimental Focus

Observing the Impact of Architecture, Activation, and Initialization.

Primary Objective

Optimizing Model Accuracy.

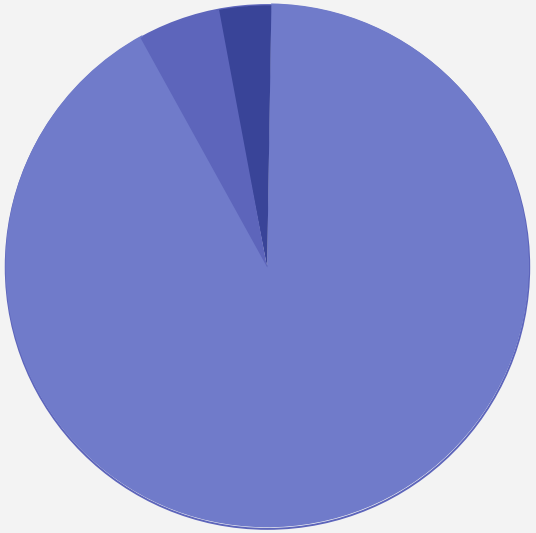
ATP Tennis Match Dataset

The model utilizes historical ATP World Tour data, spanning over five decades, to capture a rich and granular view of player performance.

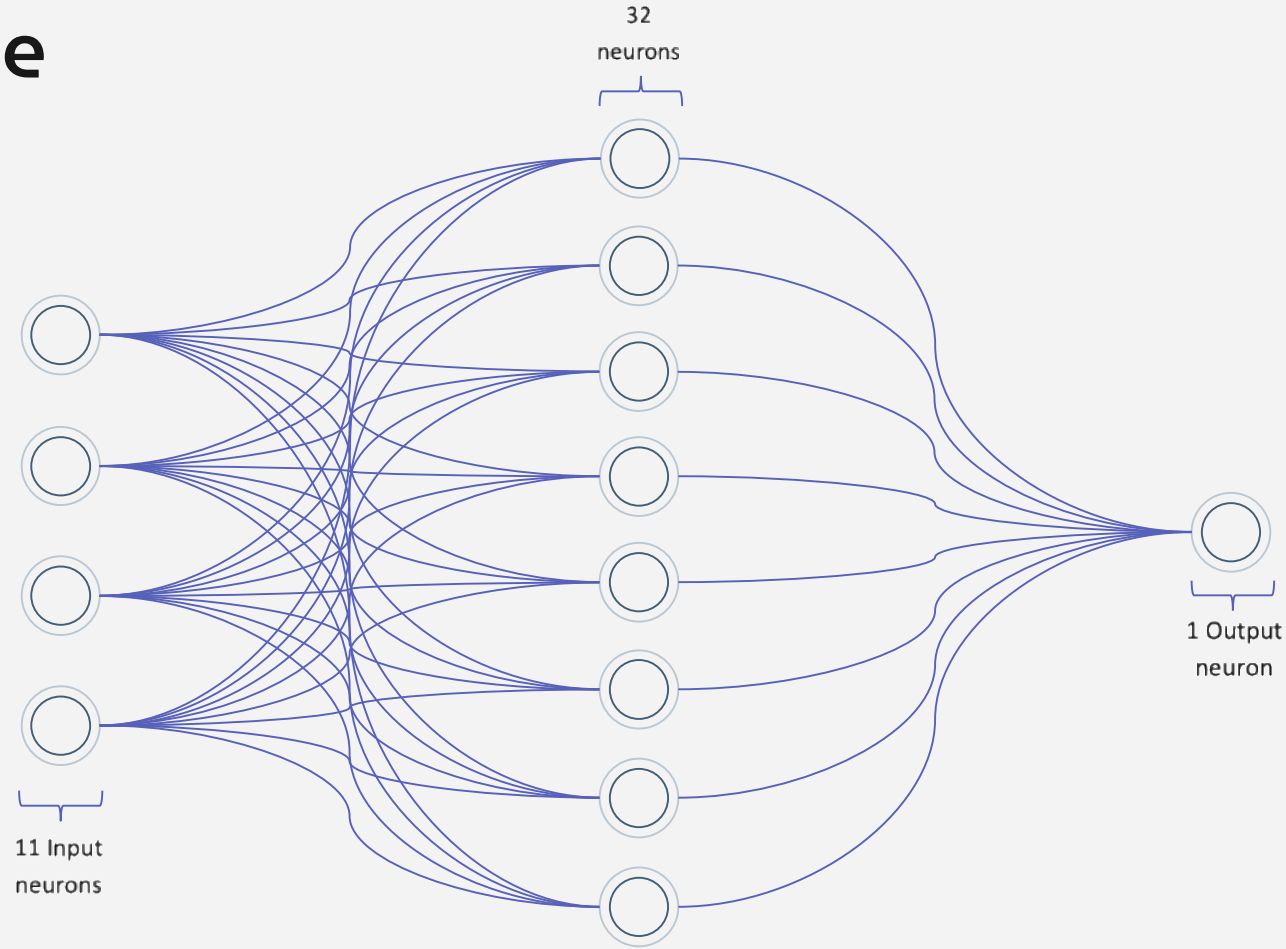
1968 - 2024 Time Horizon

194,996 Records Included

49 Key Variables



Heart Of The Model



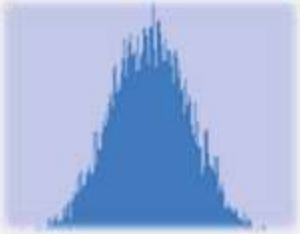
Initializing Parameters

goal: breaking the symmetry

method: xavier (glorot) initialization.

why: to preserve gradient flow for tanh activation.

process: random gaussian distribution.



Initializing Parameters

Why Not Start from Zero?

If we set them all to 0, every neuron learns the exact same thing, and the model becomes 'paralyzed.' That is why we need 'Symmetry Breaking'.

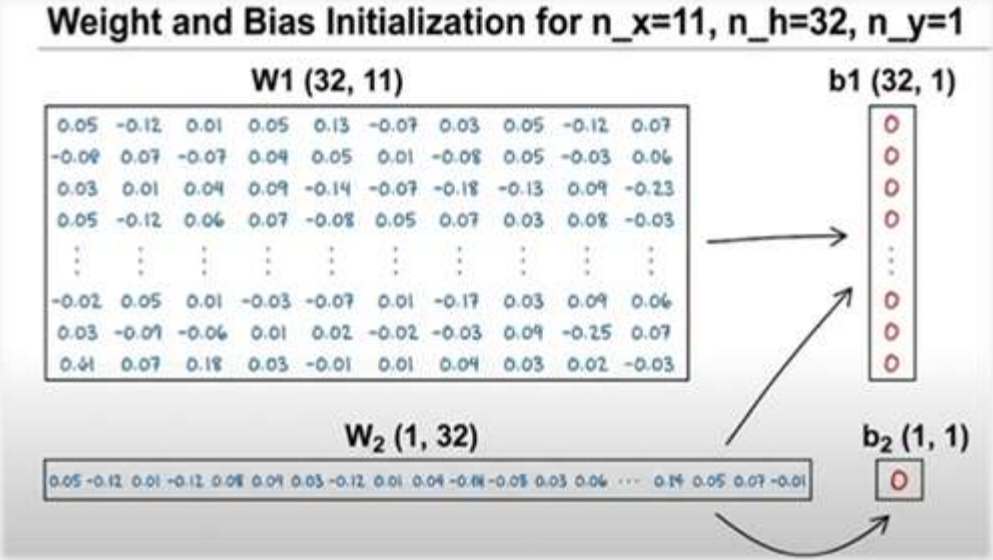
Randomness, but Controlled

We assign small random values to the weights. However, this randomness is not arbitrary

Why Xavier?

Tanh activation function is used in this model. The Xavier method balances the variance between the input size and the output signal. This prevents the 'Vanishing Gradient' problem.

Initializing Parameters



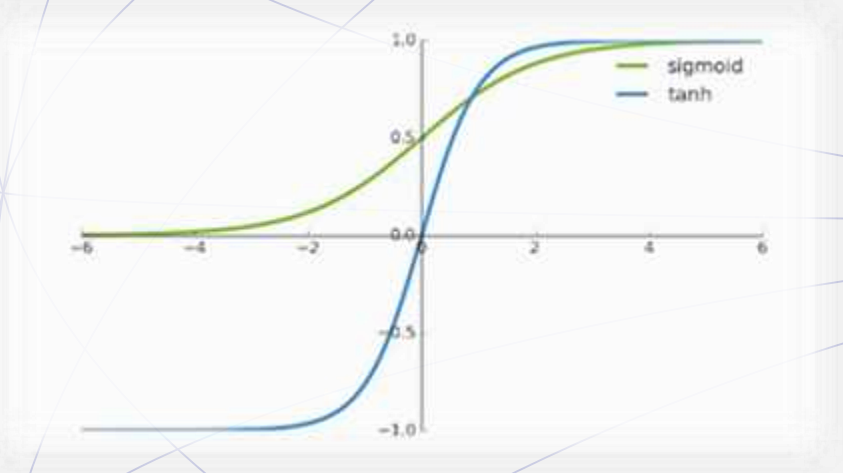

```python
def initialize_parameters(n_x, n_h, n_y):
    """
    Initializes weights and biases using the "Xavier (Glorot)" method.
    This prevents the 'vanishing gradient' problem for tanh activation.
    """

    # Incorrect Method (leads to vanishing gradients):
    # W1 = np.random.randn(n_h, n_x) * 0.01
    # W2 = np.random.randn(n_y, n_h) * 0.01

    # CORRECT METHOD (Xavier):
    # Preserve the variance by dividing by the number of incoming neurons.

    W1 = np.random.randn(n_h, n_x) * np.sqrt(1 / n_x)
    b1 = np.zeros((n_h, 1))

    W2 = np.random.randn(n_y, n_h) * np.sqrt(1 / n_h)
    b2 = np.zeros((n_y, 1))

    parameters = {"W1": W1, "b1": b1, "W2": W2, "b2": b2}
    return parameters
```

Activation Function

Structuring the Data: For the hidden layers, I specifically chose the Tanh activation function. Mathematically, it maps all inputs to a range between -1 and 1, it means our data is 'Zero-Centered'.

Why Zero-Centered Matters?: When the data is centered around zero—meaning we have both positive and negative values—it allows the gradient Descent algorithm to update the weights more efficiently. Instead of drifting in one direction'.

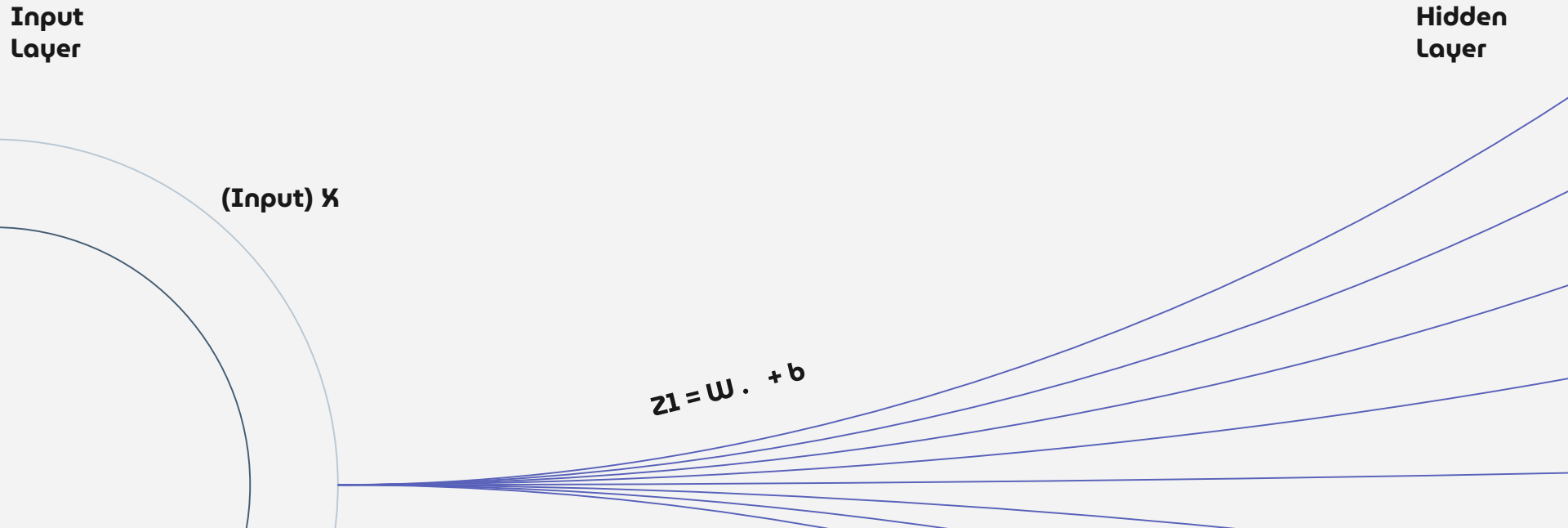
Stronger gradients: Tanh function has a very steep slope, especially around the zero point. This provides stronger gradients (derivatives) during Backpropagation.



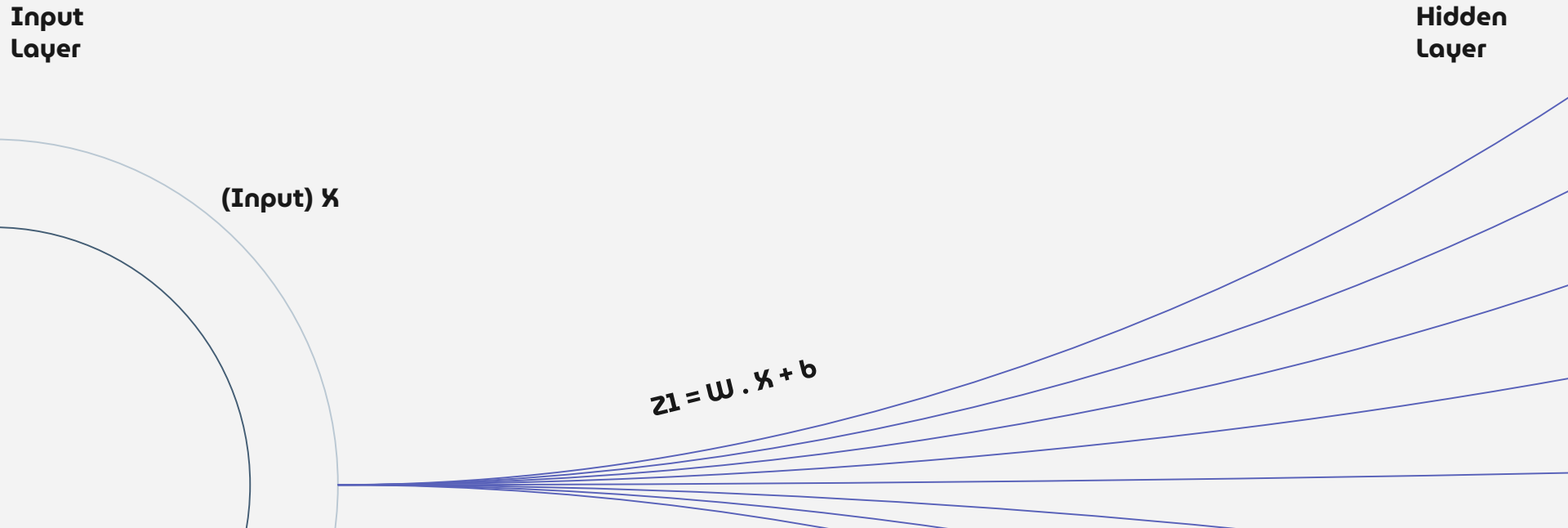
Forward Propagation

We have initialized our parameters. Now, it's time to pass the data through the network. We call this Forward Propagation.

Forward Propagation



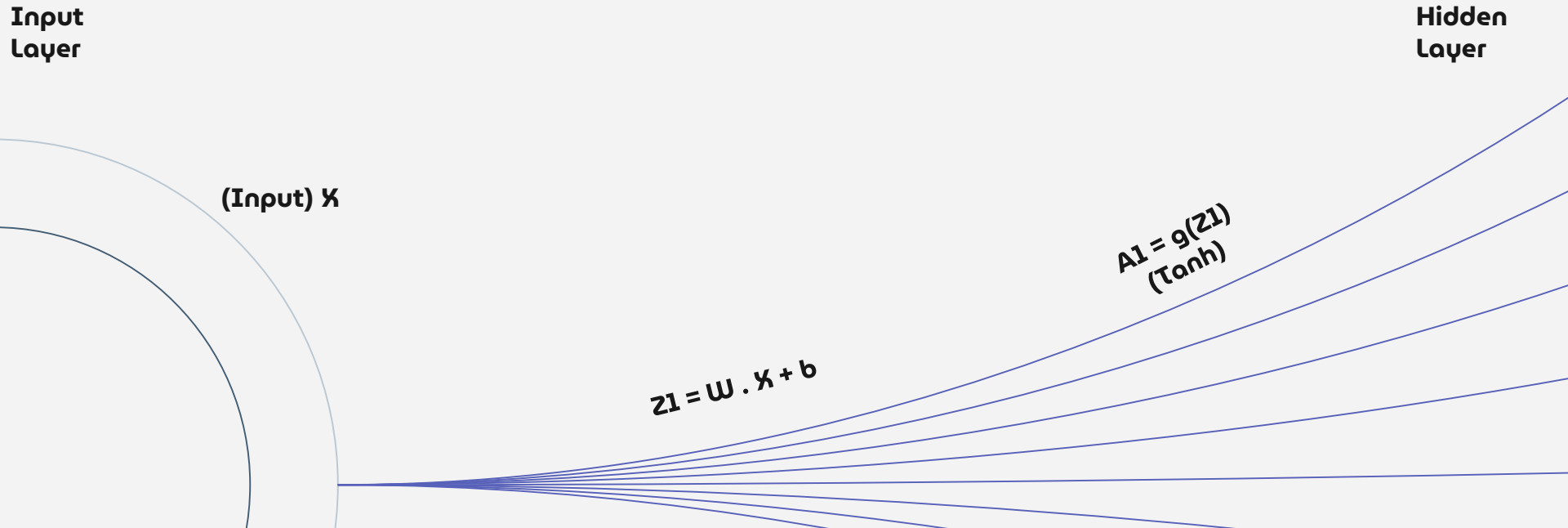
Forward Propagation



Forward Propagation

A (Activation Function, g() Tanh)

W, b (Weight and bias that we've initialized before)



Forward Propagation

Hidden Layer

Output Layer

A1

Forward Propagation

Hidden Layer

Output Layer

A1

$$Z2 = W \cdot A1 + b$$

Forward Propagation

Hidden Layer

Output Layer

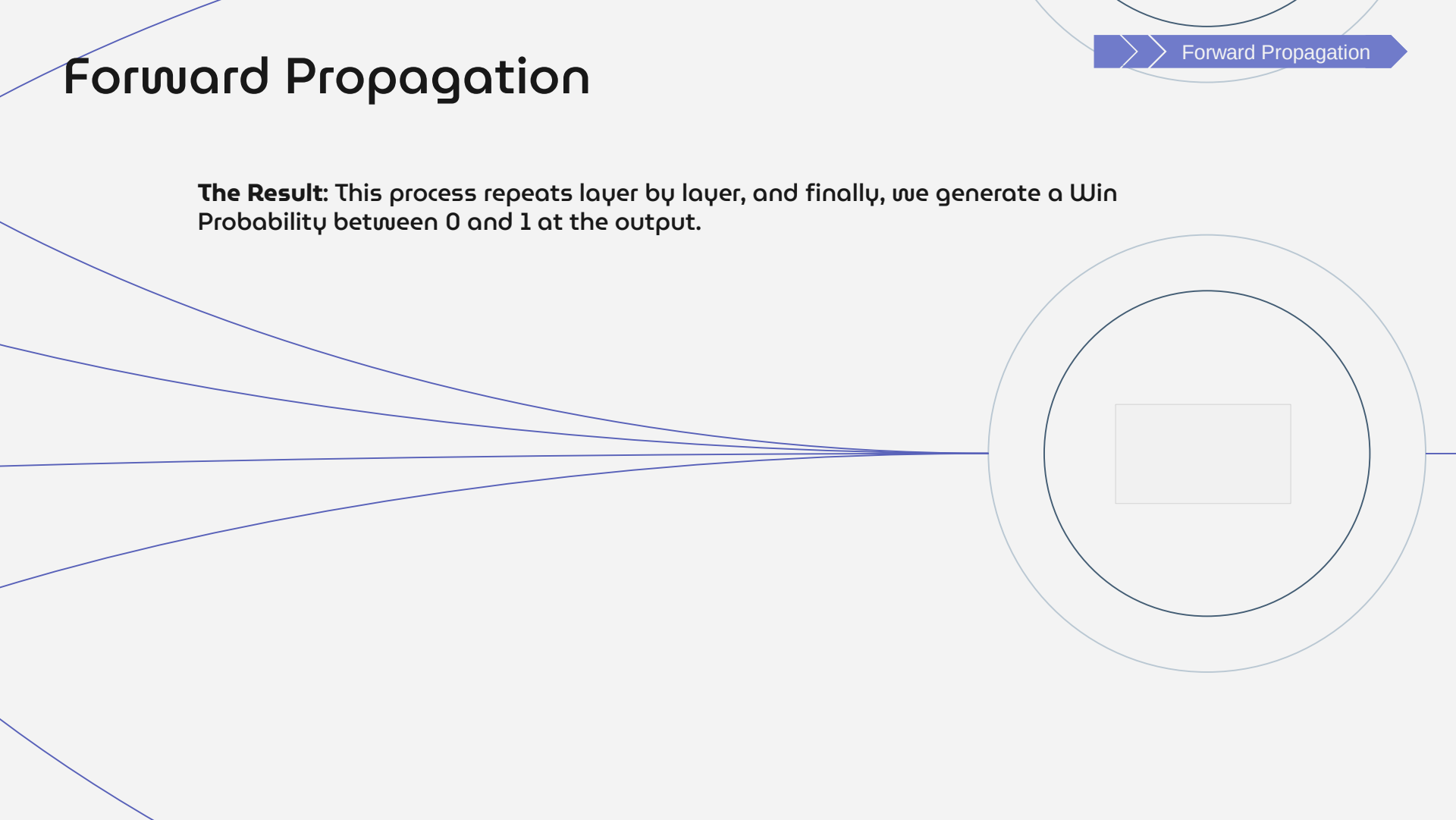
A1

Z2 = W . A1 + b

A2 = g(Z2)
(Sigmoid)

Forward Propagation

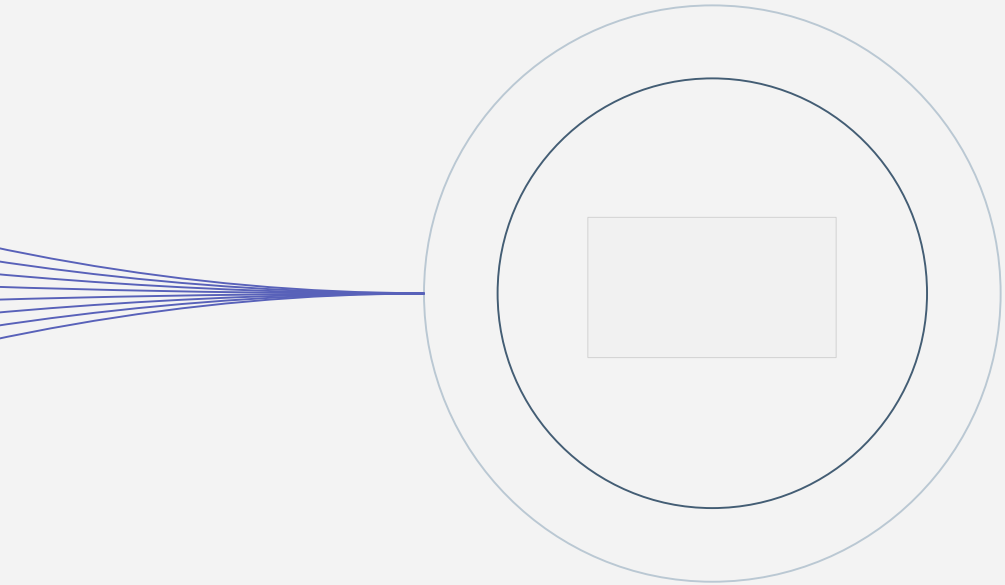
The Result: This process repeats layer by layer, and finally, we generate a Win Probability between 0 and 1 at the output.



Forward Propagation


```python
def forward_propagation(X, parameters):
    """Input data and calculates the predicted output."""
    W1 = parameters["W1"]
    b1 = parameters["b1"]
    W2 = parameters["W2"]
    b2 = parameters["b2"]

    # 1. Layer (Hidden Layer)
    # Z1 = W1 * X + b1
    Z1 = np.dot(W1, X) + b1
    A1 = np.tanh(Z1)

    # 2. Layer (Output Layer)
    # Z2 = W2 * A1 + b2
    Z2 = np.dot(W2, A1) + b2
    A2 = sigmoid(Z2) # Prediction (Y_hat)

    cache = {"Z1": Z1, "A1": A1, "Z2": Z2, "A2": A2}
    return A2, cache
```

Compute Cost



Binary Cross-Entropy Cost

goal is to measure the divergence between prediction and reality.

Mechanism: Logarithmic Penalty.

Correct Prediction -> Low Cost.

Confident Wrong Prediction -> Very High Cost.

Why Not MSE?: To ensure a "Convex" optimization surface (guarantees finding the global minimum).

Compute Cost


```python
# D) COST FUNCTION
def compute_cost(A2, Y):
    """
    Computes Binary Cross-Entropy cost.

    A2: Network predictions (Y_hat)
    Y: True Labels
    """
    m = Y.shape[1] # Number of examples (number of columns)

    epsilon = 1e-8 # Very small number
    A2_clipped = np.clip(A2, epsilon, 1 - epsilon)

    # Compute the two main terms of the Cross-Entropy formula:
    # Term 1: Y * log(A2_clipped)
    log_probs = Y * np.log(A2_clipped)

    # Term 2: (1 - Y) * log(1 - A2_clipped)
    log_probs_complement = (1 - Y) * np.log(1 - A2_clipped)

    # Total Cost J = -(1/m) * sum of all elements in (term1 + term2) matrix
    cost = - (1 / m) * (np.sum(log_probs + log_probs_complement))

    # The cost should be returned as a single number (scalar).
    cost = np.squeeze(cost)

    return cost
```

Backward Propagation

Calculating gradients (dW, db)

How should we adjust the weights to decrease the errors?

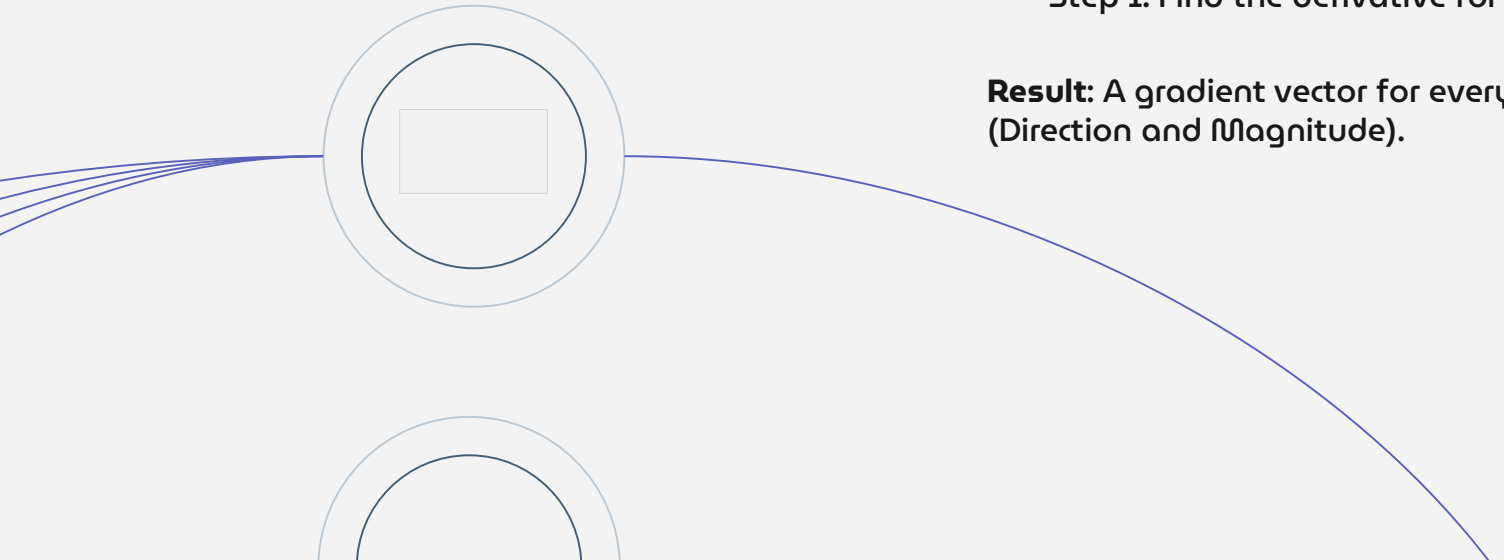
Mechanism: The chain rule.

Step 1: Calculate error ar the output.

Step 2: Distribute error back to hidden layer.

Step 1: Find the derivative for each weight.

Result: A gradient vector for every parameter (Direction and Magnitude).



Backward Propagation

$$\underbrace{\frac{\partial J}{\partial W}}_{\text{Gradient } (dW)} = \underbrace{\frac{\partial J}{\partial A}}_{\text{Error} \to \text{Output}} \cdot \underbrace{\frac{\partial A}{\partial Z}}_{\text{Output} \to \text{Hidden}} \cdot \underbrace{\frac{\partial Z}{\partial W}}_{\text{Hidden} \to \text{Weight}}$$

```python
# 2. Output Layer Gradients (Layer 2)

# dZ2 = A2 - Y (Simplified formula derived from Cross-Entropy + Sigmoid combination)
dZ2 = A2 - Y

# dW2 = (1/m) * dZ2 . A1^T
dW2 = (1 / m) * np.dot(dZ2, A1.T)

# db2 = mean of dZ2 over the examples
db2 = np.mean(dZZ, axis=1, keepdims=True)

# 3. Hidden Layer Gradients (Layer 1)

# Backpropagating the error: W2^T . dZ2
# This distributes the error from W2 back to the neurons in Layer 1.
dZ1_weighted = np.dot(W2.T, dZ2)

# tanh derivative: 1 - A1^2
dZ1 = dZ1_weighted * (1 - np.power(A1, 2))

# dW1 = (1/m) * dZ1 . X^T
dW1 = (1 / m) * np.dot(dZ1, X.T)

# db1 = mean of dZ1 over the examples
db1 = np.mean(dZ1, axis=1, keepdims=True)

# Collect gradients in a dictionary
grads = {"dw1": dw1, "db1": db1, "dw2": dw2, "db2": db2}

return grads
```

Updating Parameters

gradient descent

Reach the Global Minimum (Lowest Cost).

Key Parameter: Learning Rate (α).

Controls the step size.

$$W_{new} = W_{old} - \alpha \cdot dW$$

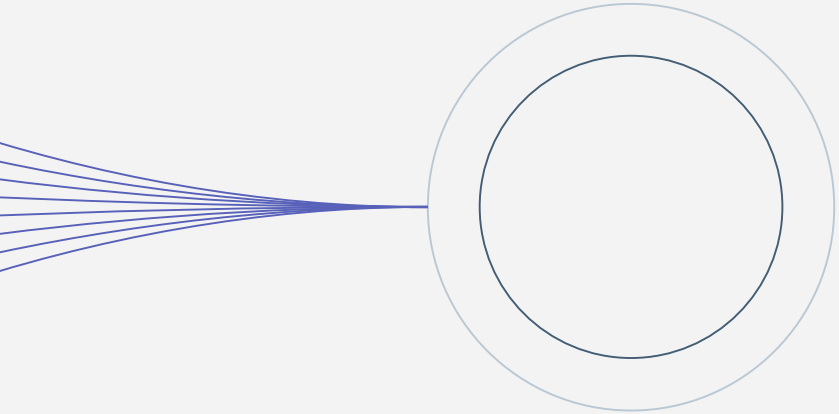
Too High -> Overshoots the target (diverges).

Too Low –> Takes forever to converge.

Updating Parameters


```python
# F) PARAMETER UPDATE (GRADIENT DESCENT)
def update_parameters(parameters, grads, learning_rate):

    # Retrieve parameters
    W1 = parameters["W1"]
    b1 = parameters["b1"]
    W2 = parameters["W2"]
    b2 = parameters["b2"]

    # Retrieve gradients
    dw1 = grads["dw1"]
    db1 = grads["db1"]
    dw2 = grads["dw2"]
    db2 = grads["db2"]

    # Apply Gradient Descent Rule
    W1 = W1 - learning_rate * dw1
    b1 = b1 - learning_rate * db1
    W2 = W2 - learning_rate * dw2
    b2 = b2 - learning_rate * db2

    # Save updated parameters in a dictionary
    parameters = {"W1": W1, "b1": b1, "W2": W2, "b2": b2}

    return parameters
```

Forward Propagation For Validation



Validation

The reason we do this is to measure the risk of Overfitting. The model might have memorized the training data perfectly, but how will it perform on the 2022 data it has never seen before?

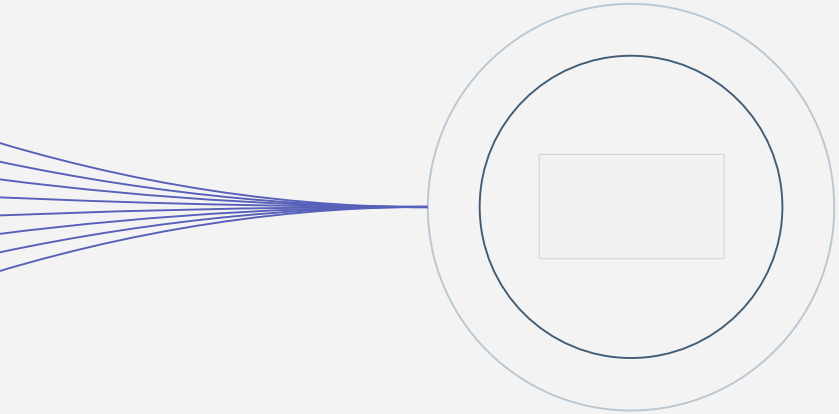
Forward propagation only

Rule 1: No backward propagation

Rule 2: No parameter updates

goal is to detect overfitting

Compute Cost For Validation



Same ruler, Different data

When measuring the error on the validation set, we use the exact same mathematical formula (Binary Cross-Entropy) that we used for training.

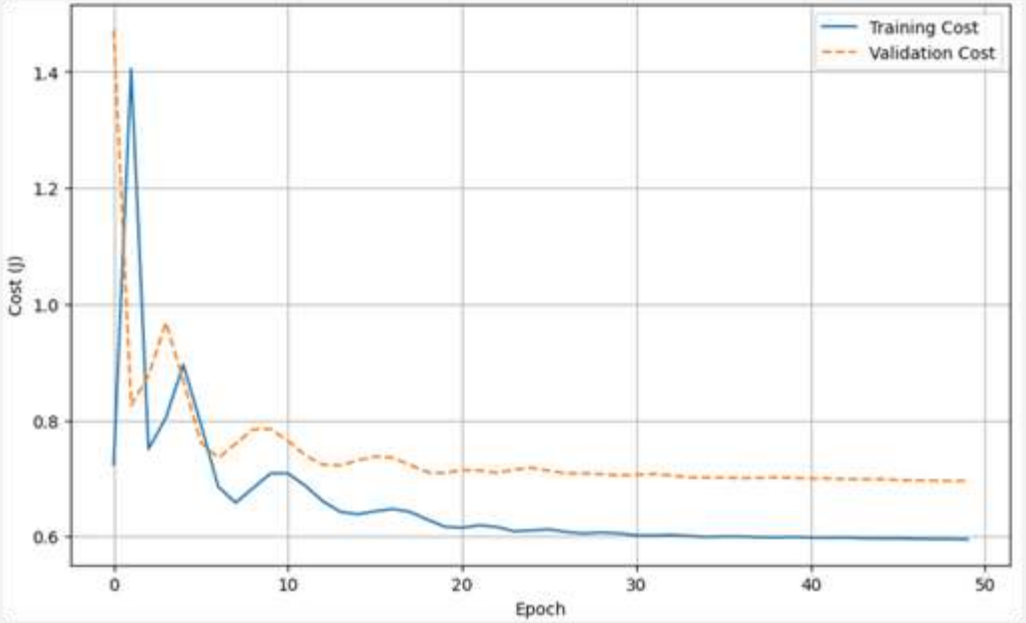
goal is to compare (J1 (train) vs J2 (val)).

The Critical Question

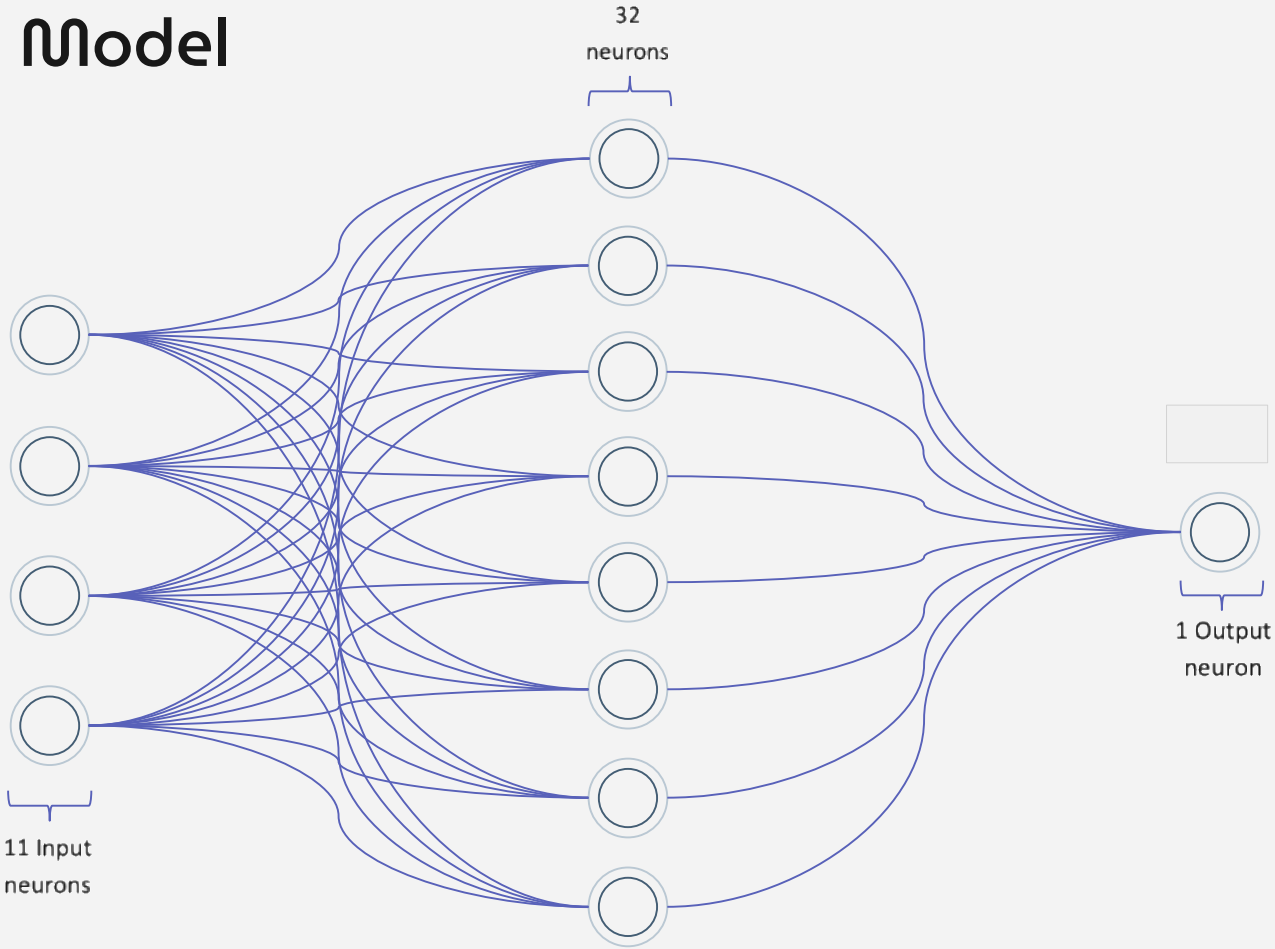
Is the model learning the pattern?

Or is it just memorizing the data?

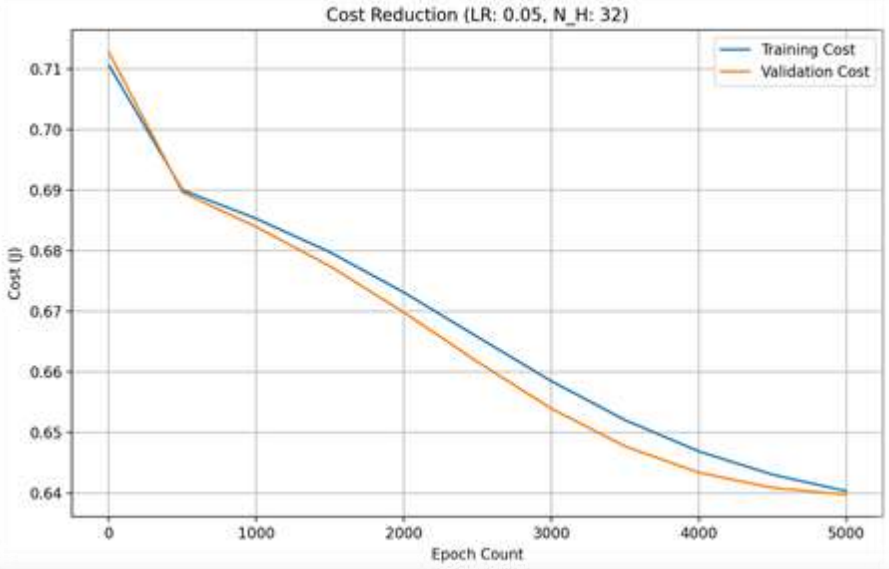
Compute Cost For Validation

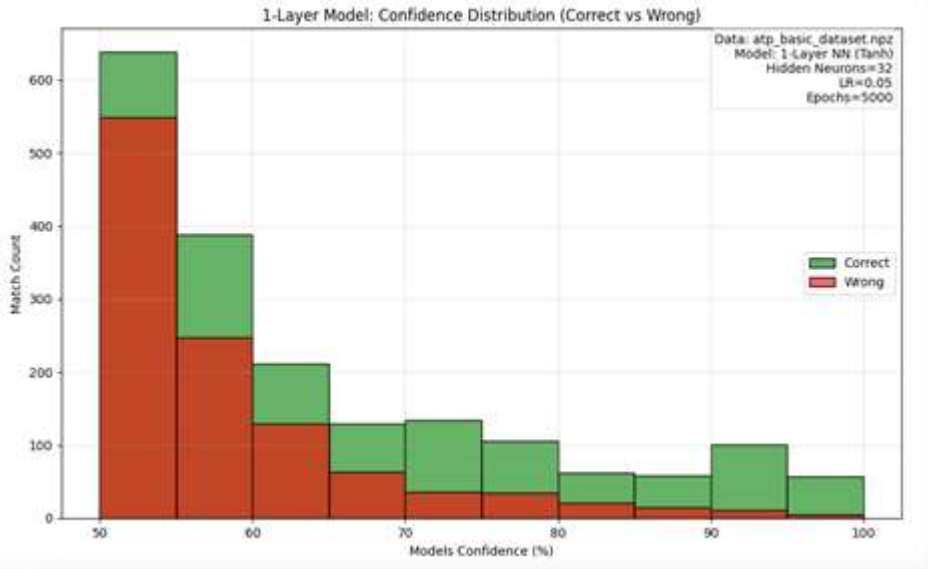


Improving the Model



Improving the Model





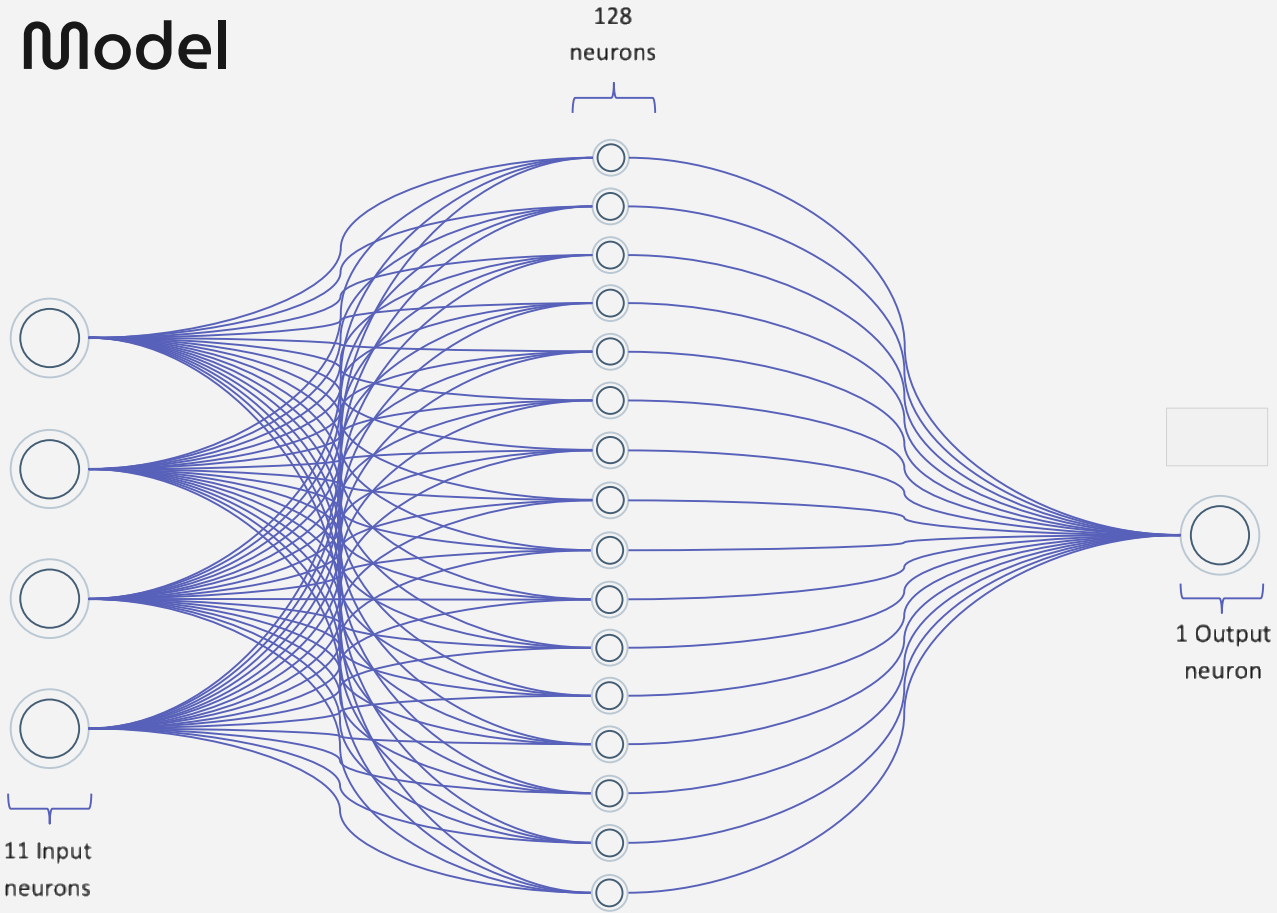
62.90%

Test Accuracy

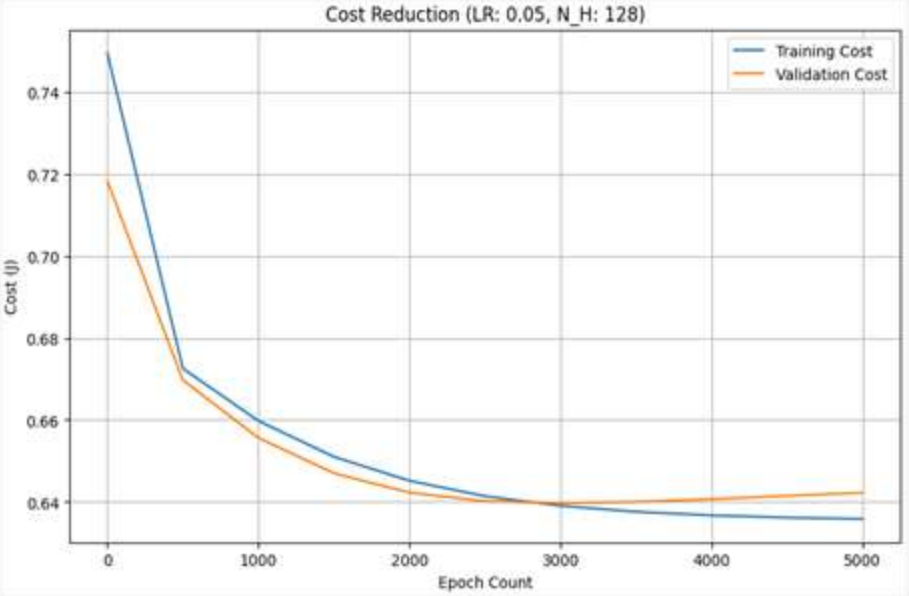
417 parameters

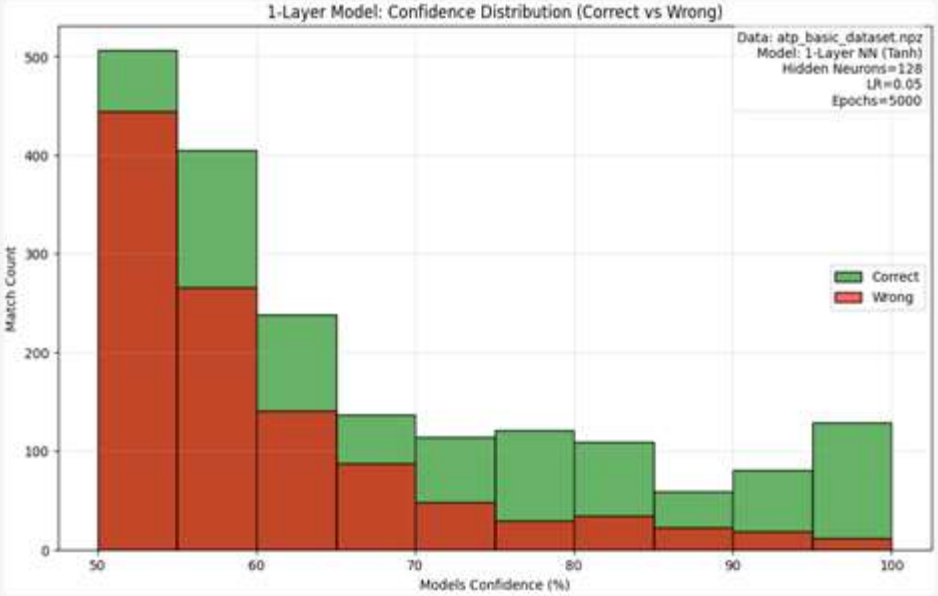
2 min 40 sec ~

Improving the Model



Improving the Model



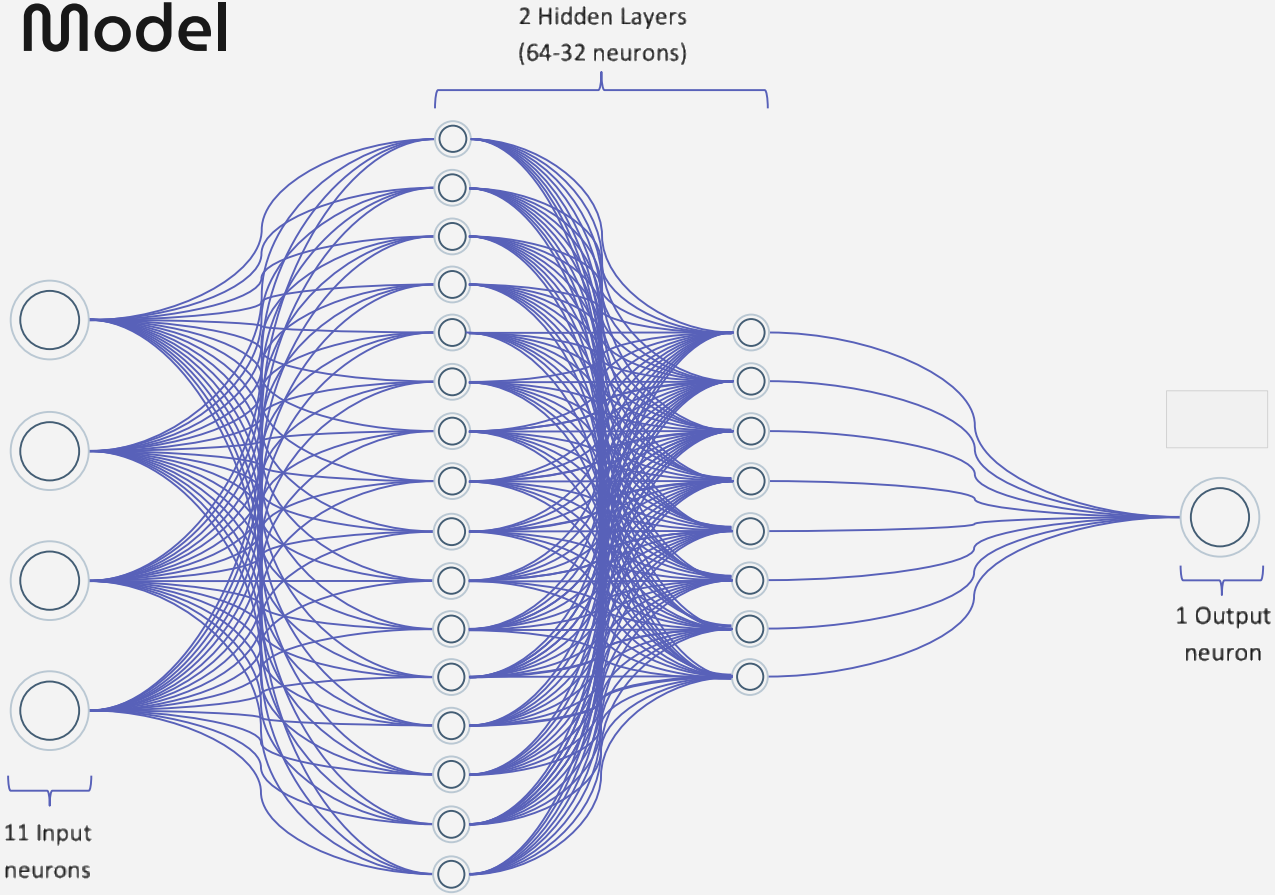


↑ **63.30%**
Test Accuracy

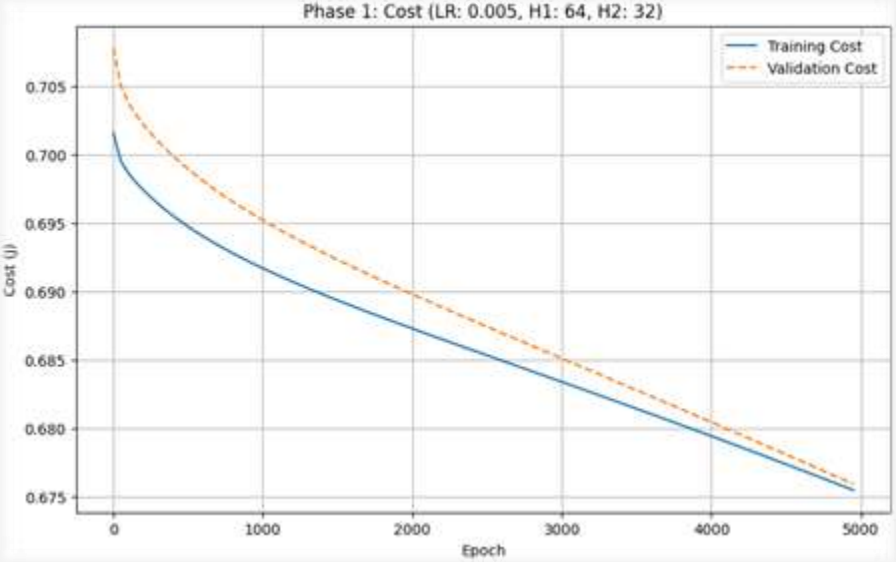
↑
1,665 parameters

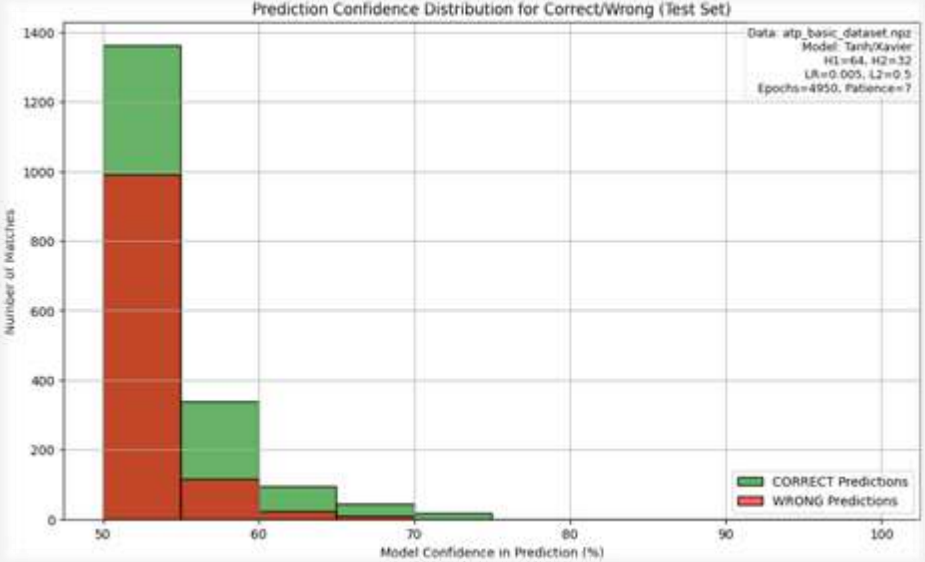
↑
10 min ~

Improving the Model



Improving the Model



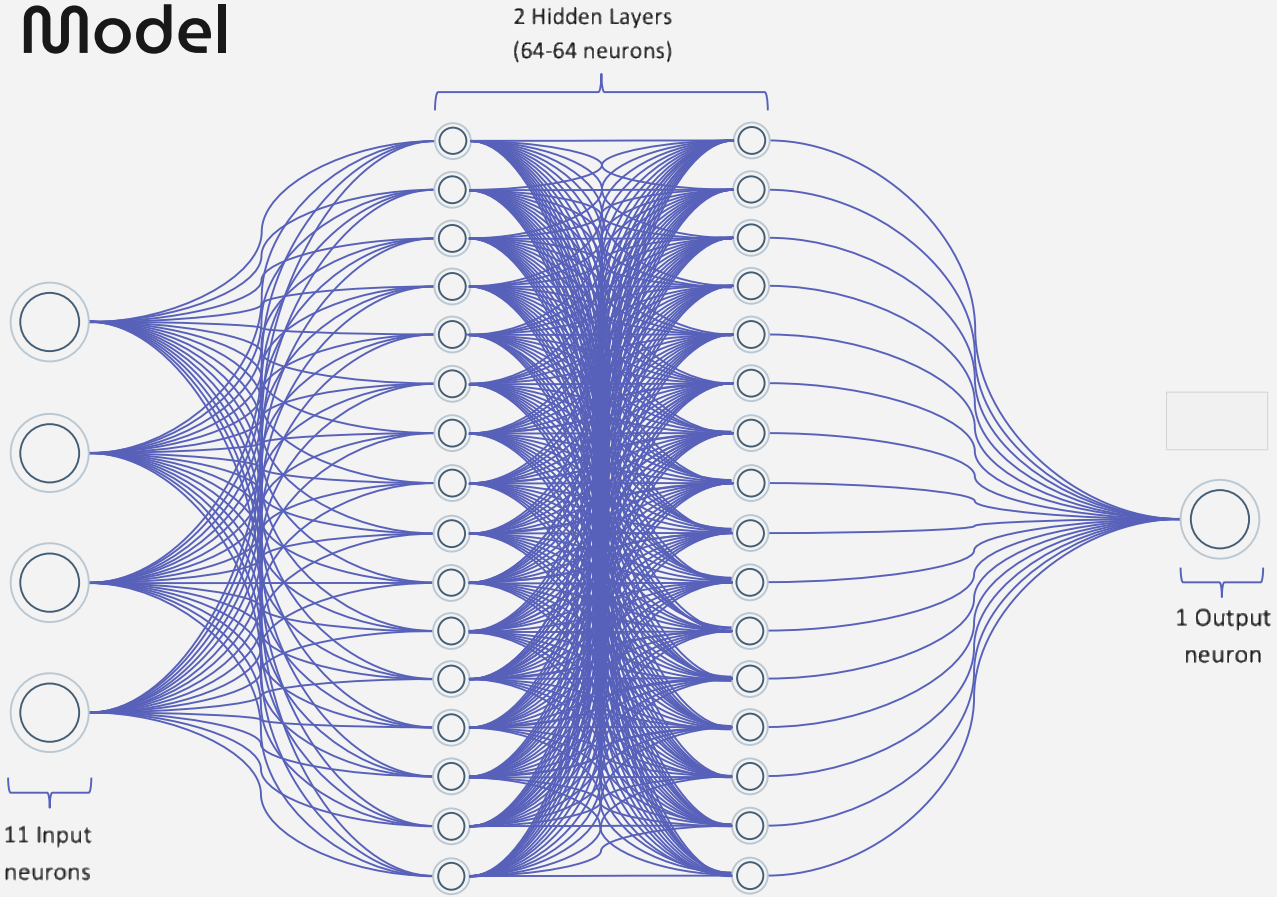


61.30%
Test Accuracy

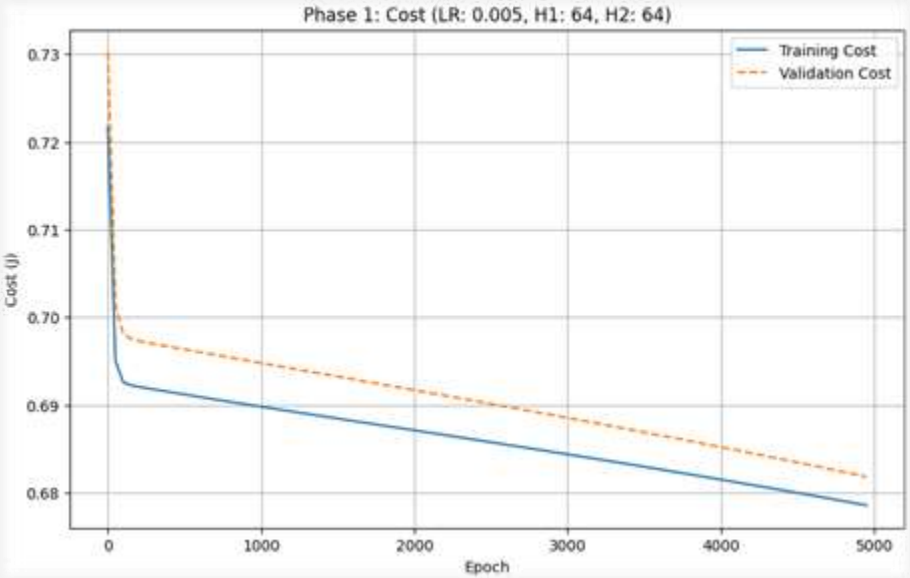
2,881 parameters

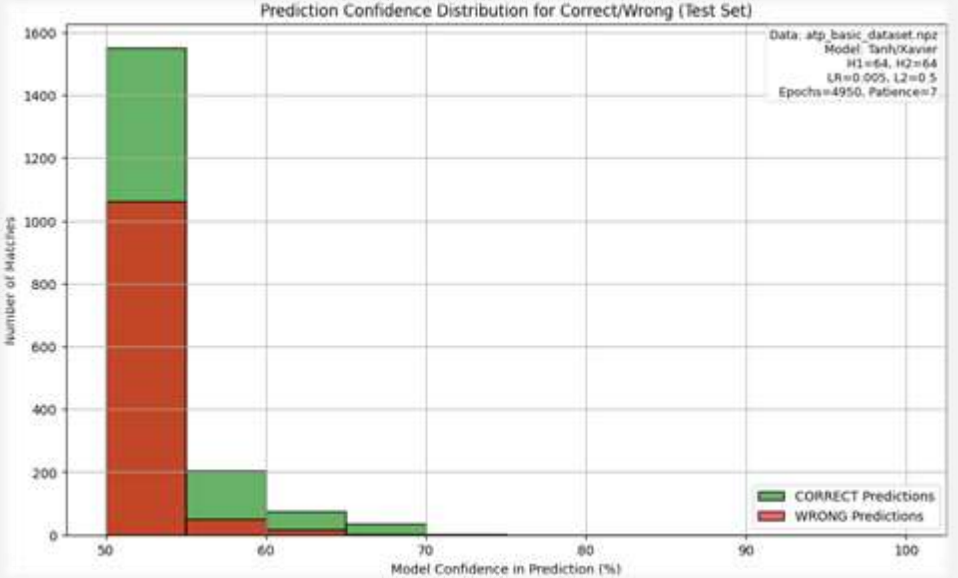
8 min 40 sec ~

Improving the Model



Improving the Model





↑ 62.60%
Test Accuracy

↑ 4,993 parameters

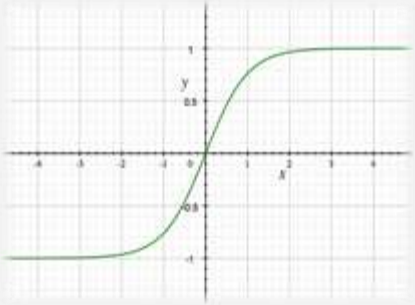
↑ 19 min ~

Improving the Model

Before

Activation: Tanh

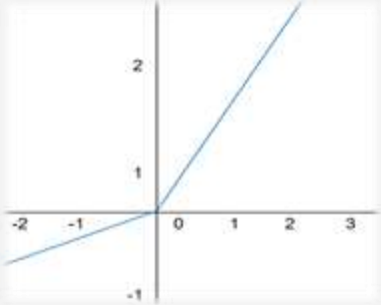
Initialization: Xavier

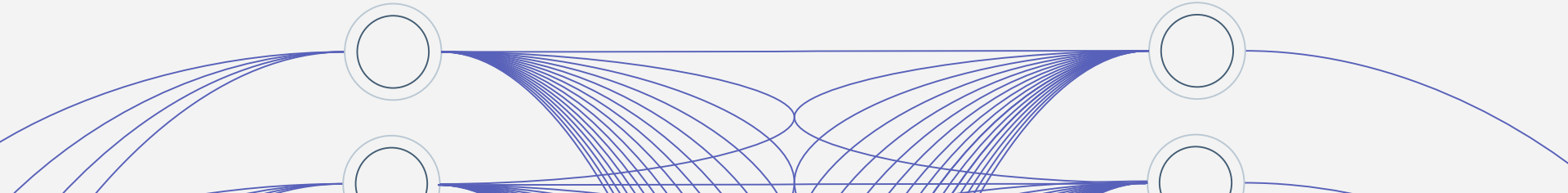


After

Activation: Leaky Relu

Initialization: He





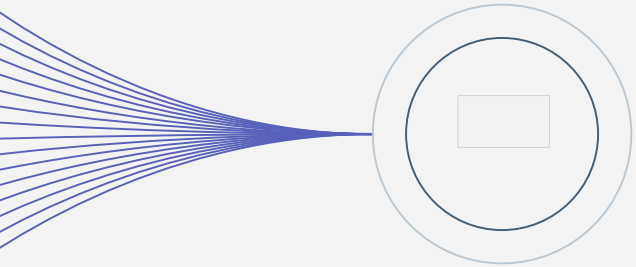
Improving the Model

L2 Regularization

Our model became so capable that it started 'memorizing' the data (Overfitting) rather than learning it.

Operation: Add the square of weights to the cost function.

By adding the «lambd» parameter to the code, I imposed a 'Complexity Tax' on the model.



Early Stopping

We've added a safety mechanism called Early Stopping, goal is to save the best model before Overfitting begins.

Mechanism: Monitor Validation Error.

If $J_{train} \downarrow$ but $J_{val} \uparrow \rightarrow$ **STOP!**

Key Parameter (Patience): Tolerance level for no improvement.

we might define 5,000 epochs and wait until the end. But often, the model learns everything it needs by epoch 1,000. Continuing to train beyond that point doesn't help; in fact, it hurts.

Improving the Model

```python
def compute_cost(A_out, Y, parameters, lambd=0):
    """
    Computes Binary Cross-Entropy and generic L2 regularization penalty.
    A_out: Final activation of the network (A2, A3, etc.)
    """
    m = Y.shape[1]

    # 1. Cross-Entropy Cost
    epsilon = 1e-8
    A_clipped = np.clip(A_out, epsilon, 1 - epsilon)
    log_probs = Y * np.log(A_clipped)
    log_probs_complement = (1 - Y) * np.log(1 - A_clipped)
    cross_entropy_cost = - (1 / m) * (np.sum(log_probs + log_probs_complement))

    # 2. L2 Regularization Penalty (Generic)
    # Applies to all weights (W1, W2, W3...) in the parameters dictionary
    L2_cost = 0
    L = len(parameters) // 2  # Number of layers (W,b pairs)
    for l in range(1, L + 1):
        L2_cost += np.sum(np.square(parameters["W" + str(l)]))

    L2_regularization_cost = (lambd / (2 * m)) * L2_cost

    cost = cross_entropy_cost + L2_regularization_cost
    cost = np.squeeze(cost)

    return cost
```

```python
    if i % print_interval == 0:
        costs.append(cost)

        if do_validation:
            A_val, _ = forward_propagation_LeakyReLU(X_val, parameters)
            cost_val = compute_cost(A_val, Y_val, parameters, lambd)
            validation_costs.append(cost_val)

            if print_cost:
                print(f"Epoch {i} | Training Cost: {cost:.4f} | Validation Cost: {cost_val:.4f}")

            if cost_val < best_val_cost:
                best_val_cost, best_parameters, best_epoch, patience_counter = cost_val, parameters.copy(), i, 0
                if print_cost: print(f"          -> (The best model has been saved.)")
                FINAL_EPOCHS = best_epoch
            else:
                patience_counter += 1
                if print_cost: print(f"          -> (Patience: {patience_counter}/{patience})")

            if patience_counter >= patience:
                print(f"\n--- Early Stopping: {best_epoch} (Validation Cost: {best_val_cost:.4f})")
                break
        else:
            validation_costs.append(np.nan)
            if print_cost: print(f"Epoch {i} | Training Cost: {cost:.4f}")

if do_validation and best_parameters:
    return best_parameters, costs, validation_costs

return parameters, costs, validation_costs
```

Improving the Model

Adam Optimizer

The reason I added the Adam Optimizer to my code is to give this process Momentum. The Adam algorithm acts like a heavy iron ball rolling down a hill. It accelerates as the slope continues downwards and brakes when it hits flat terrain.

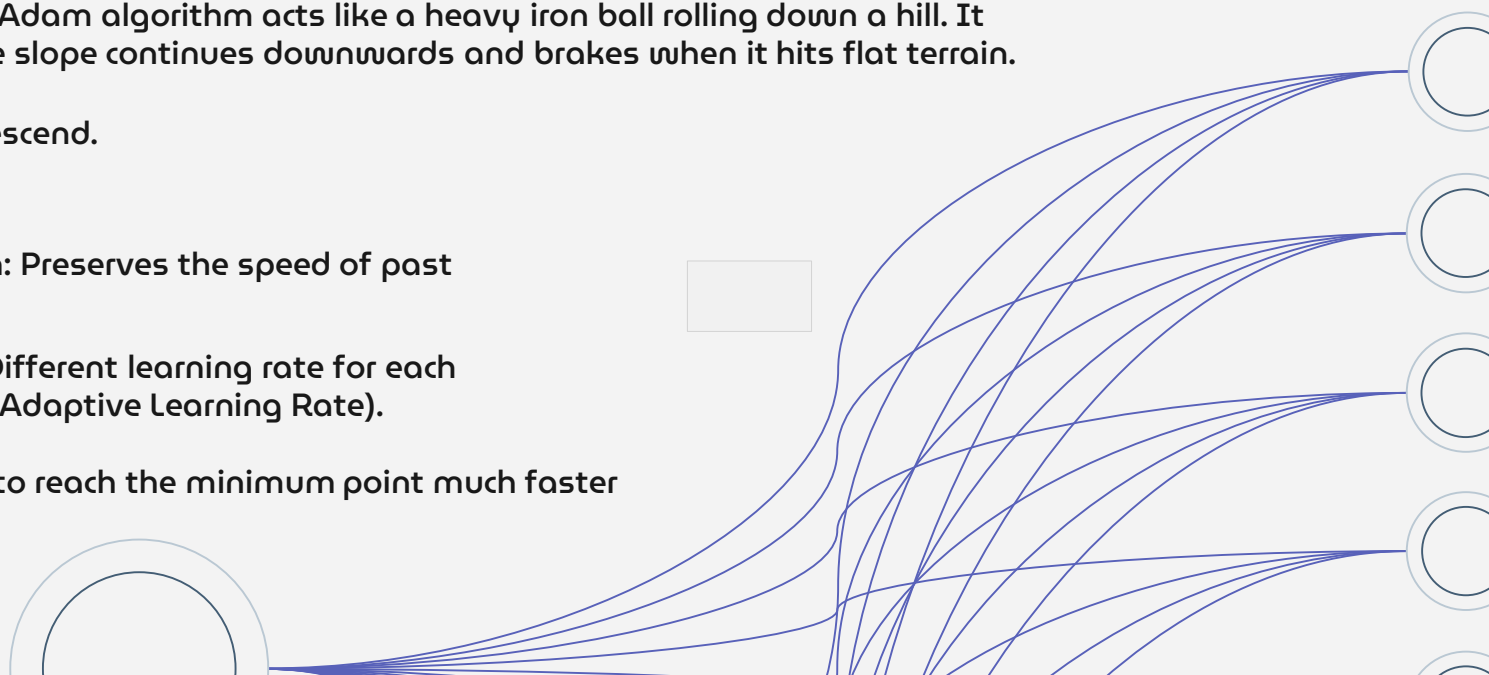
Role: Smart Descend.

Mechanism:

Momentum: Preserves the speed of past gradients.

RMSProp: Different learning rate for each parameter (Adaptive Learning Rate).

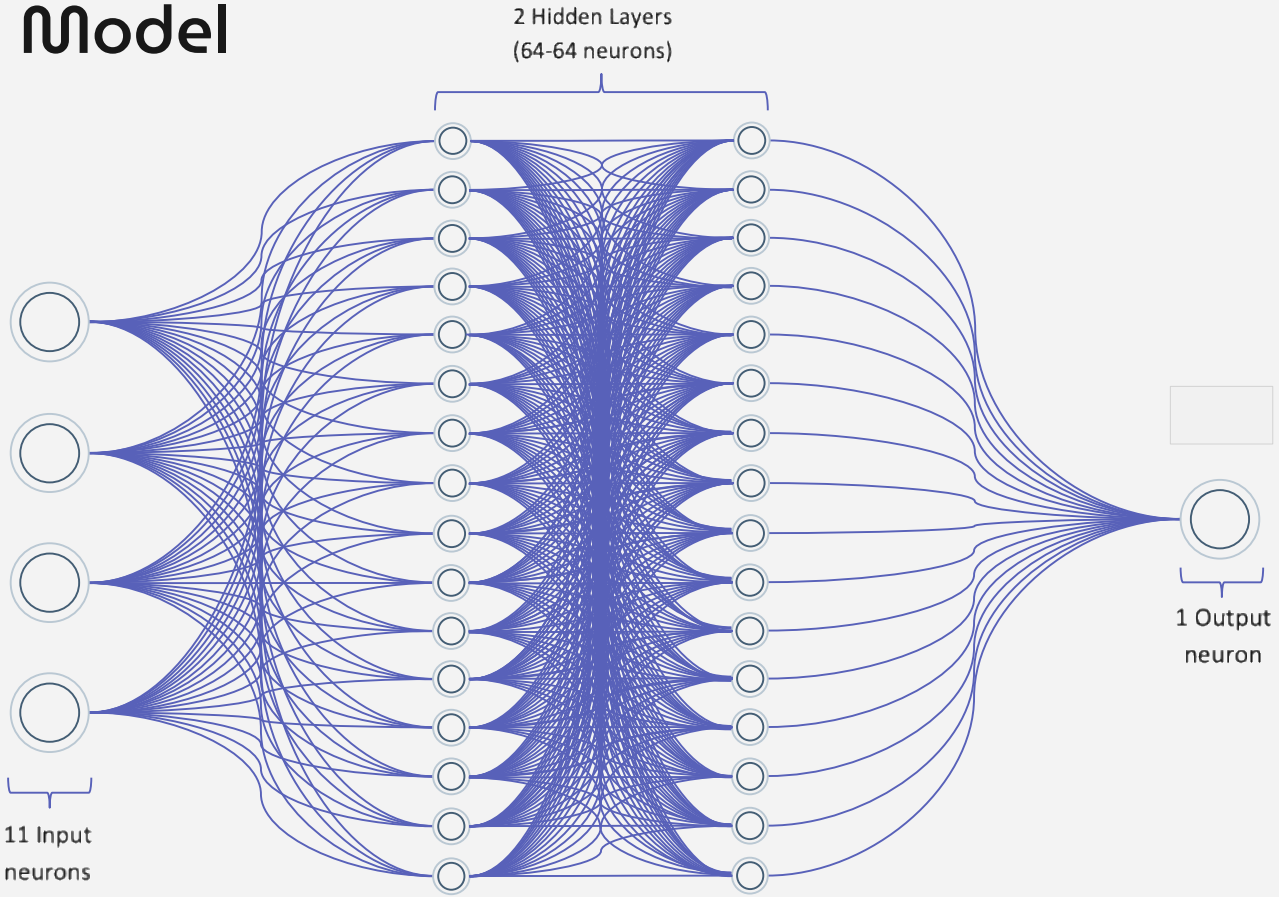
This allowed me to reach the minimum point much faster and more stable.



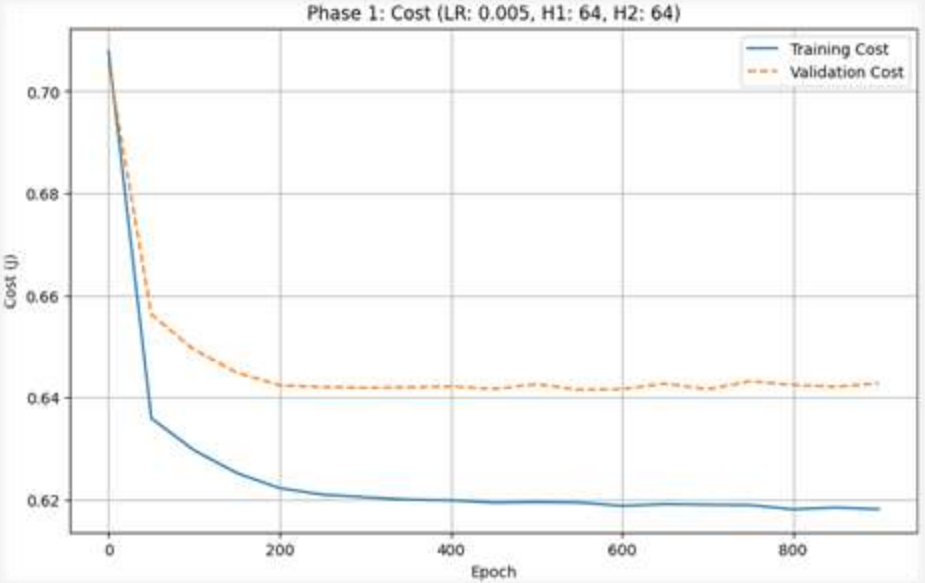
Improving the Model

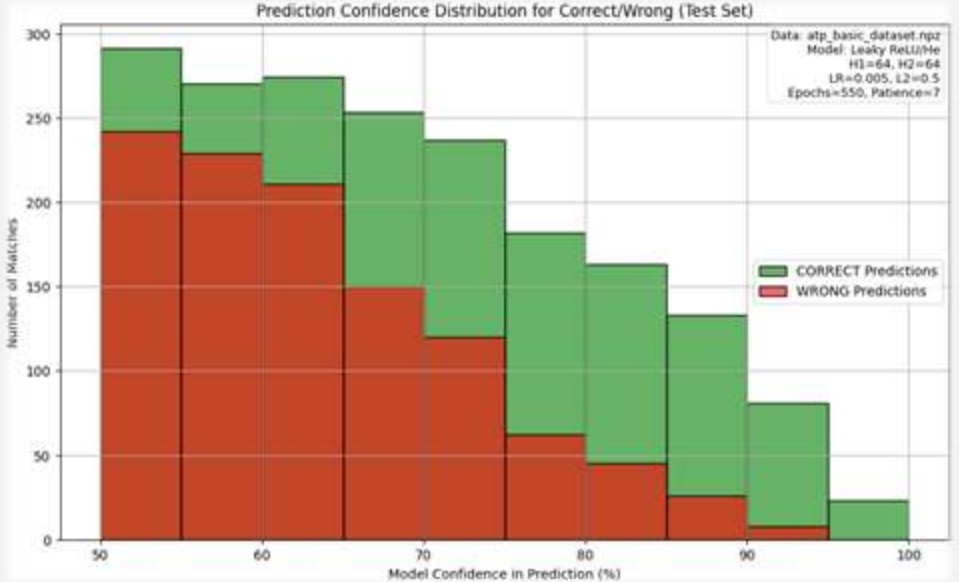
```
def update_parameters_adam(parameters, grads, v, s, t, learning_rate, beta1, beta2, epsilon):
    L = len(parameters) // 2
    v_corrected = {}
    s_corrected = {}

    for l in range(1, L + 1):
        dw = grads["dw" + str(l)]
        db = grads["db" + str(l)]

        # Update v and s
        v["dw" + str(l)] = beta1 * v["dw" + str(l)] + (1 - beta1) * dw
        v["db" + str(l)] = beta1 * v["db" + str(l)] + (1 - beta1) * db
        s["dw" + str(l)] = beta2 * s["dw" + str(l)] + (1 - beta2) * np.power(dw, 2)
        s["db" + str(l)] = beta2 * s["db" + str(l)] + (1 - beta2) * np.power(db, 2)

        # Bias Correction
        v_corrected["dw" + str(l)] = v["dw" + str(l)] / (1 - np.power(beta1, t))
        v_corrected["db" + str(l)] = v["db" + str(l)] / (1 - np.power(beta1, t))
        s_corrected["dw" + str(l)] = s["dw" + str(l)] / (1 - np.power(beta2, t))
        s_corrected["db" + str(l)] = s["db" + str(l)] / (1 - np.power(beta2, t))

        # Update parameters
        W_update = (learning_rate + v_corrected["dw" + str(l)]) / (np.sqrt(s_corrected["dw" + str(l)]) + epsilon)
        b_update = (learning_rate + v_corrected["db" + str(l)]) / (np.sqrt(s_corrected["db" + str(l)]) + epsilon)

        parameters["W" + str(l)] = parameters["W" + str(l)] - W_update
        parameters["b" + str(l)] = parameters["b" + str(l)] - b_update

    return parameters, v, s
```

Improving the Model



Improving the Model



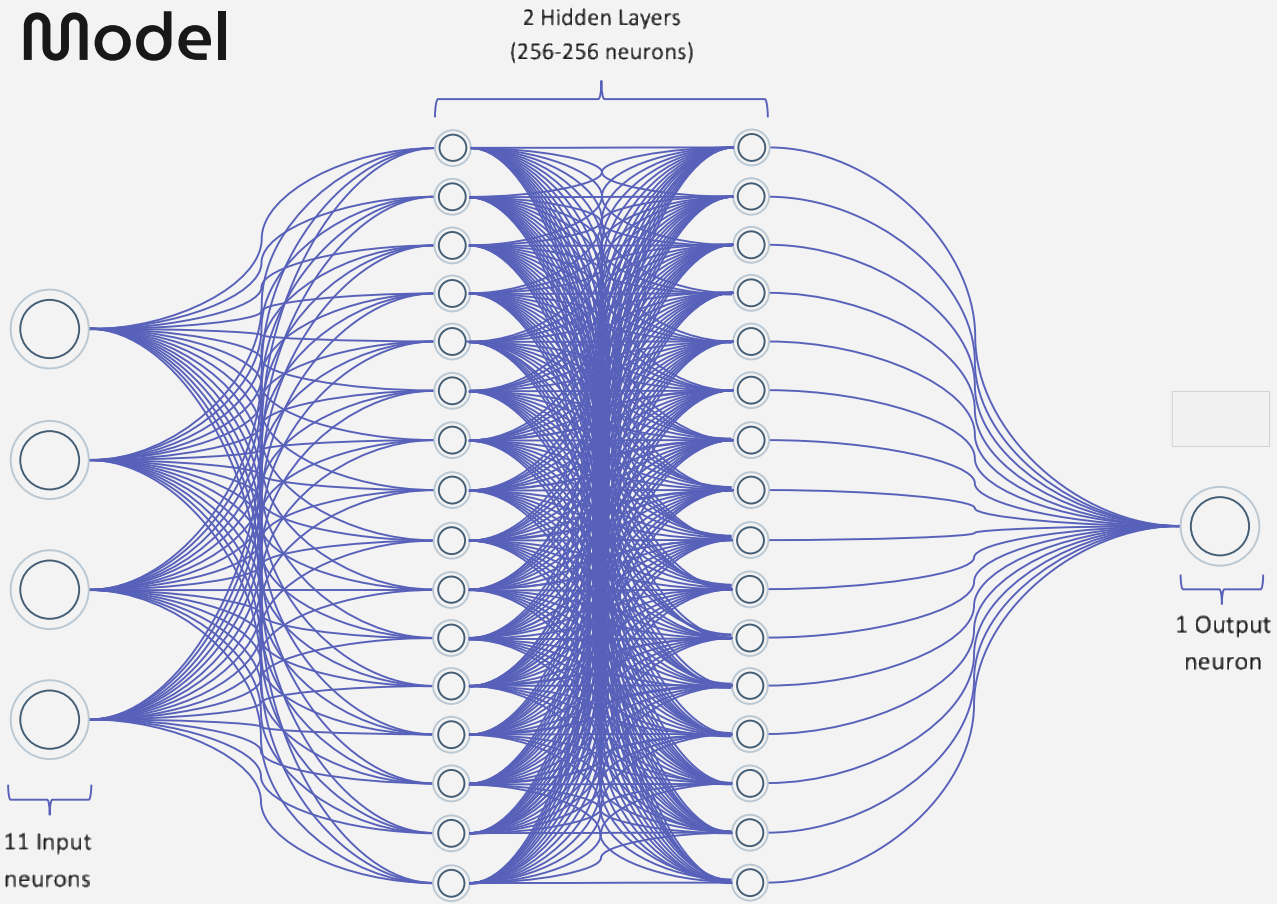


↑ **63.40%**
Test Accuracy

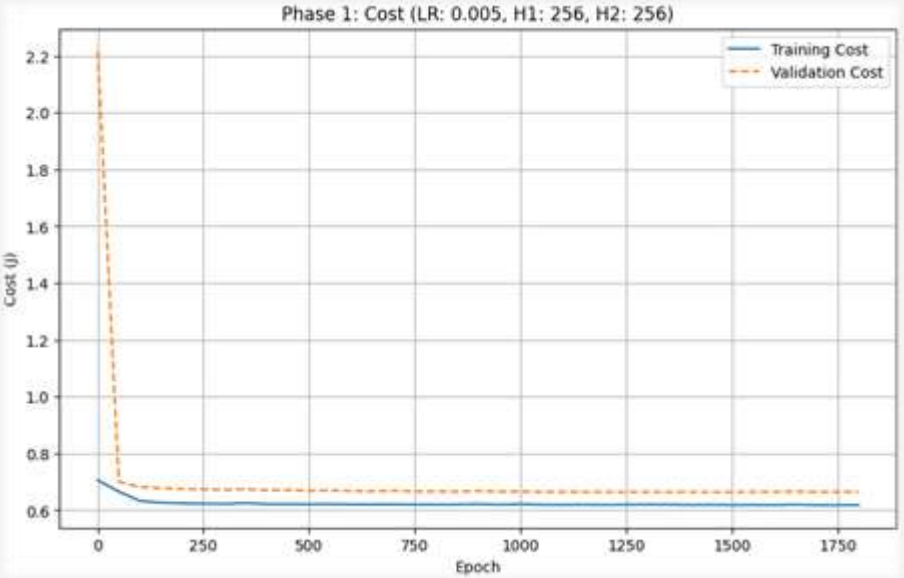
–
4,993 parameters

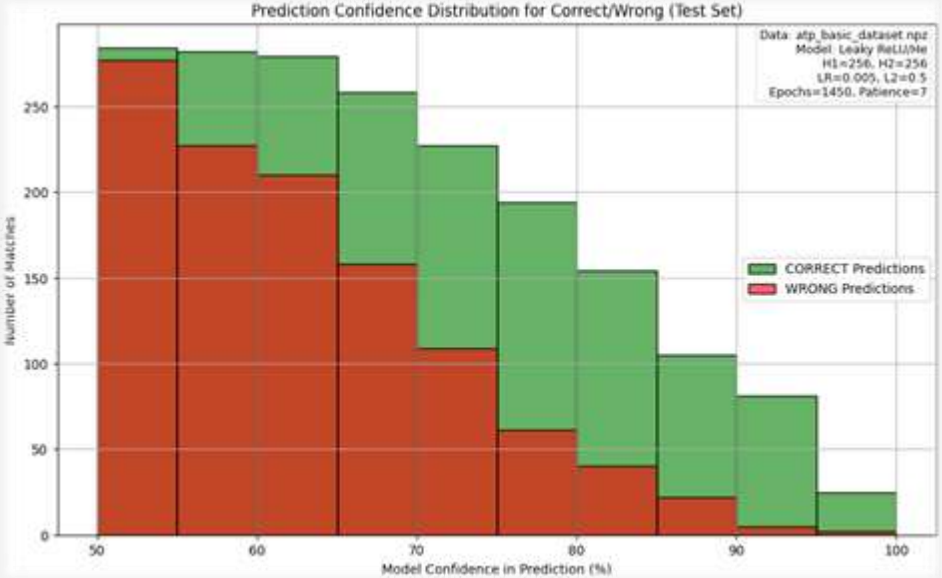
↓
1 min 30 sec ~

Improving the Model



Improving the Model



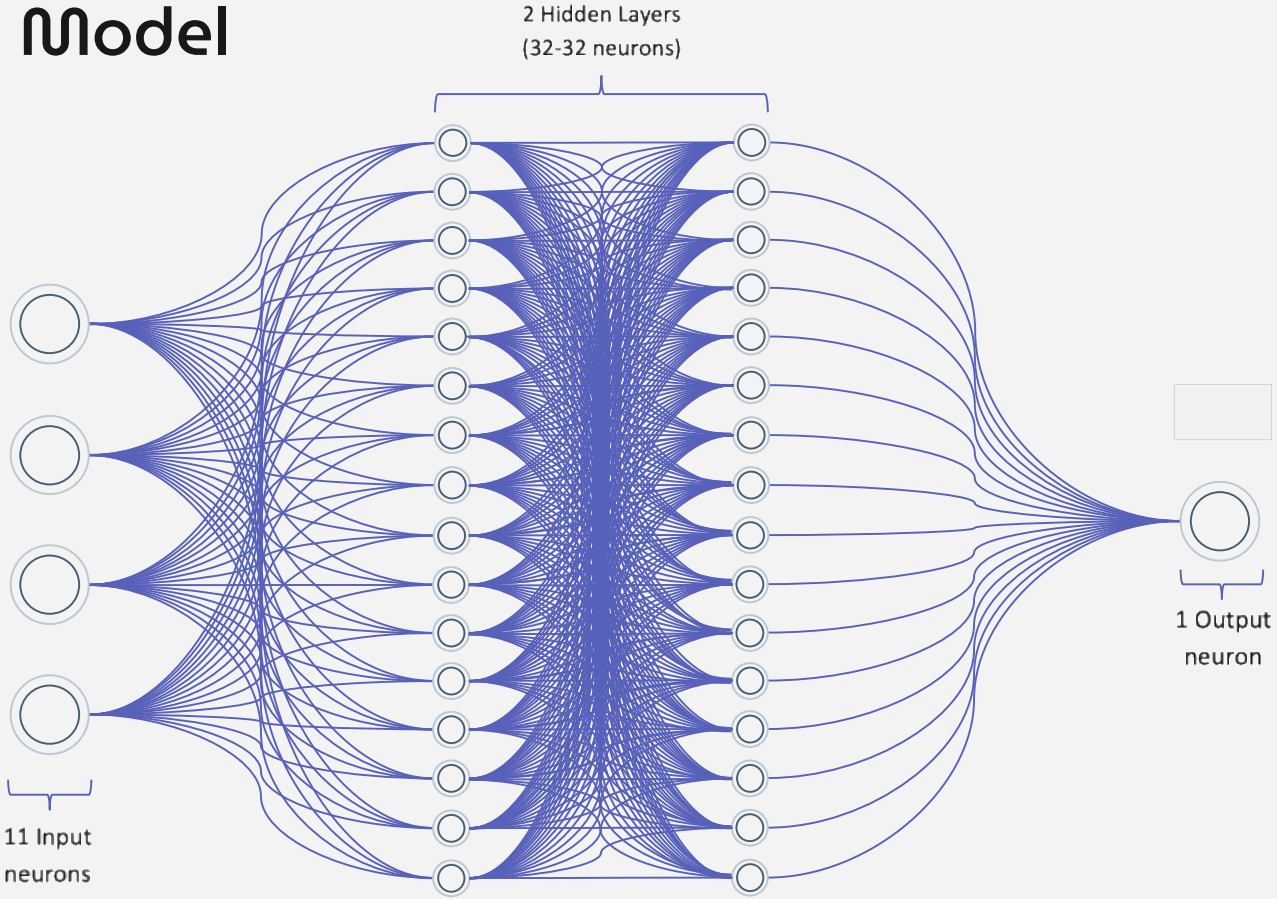


↓ **63.05%**
Test Accuracy

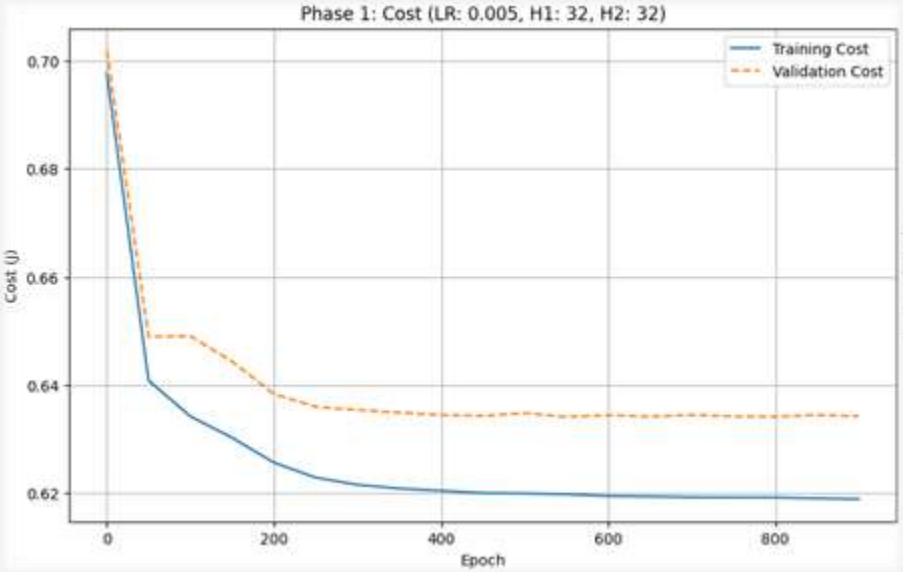
↑ 69,121 parameters

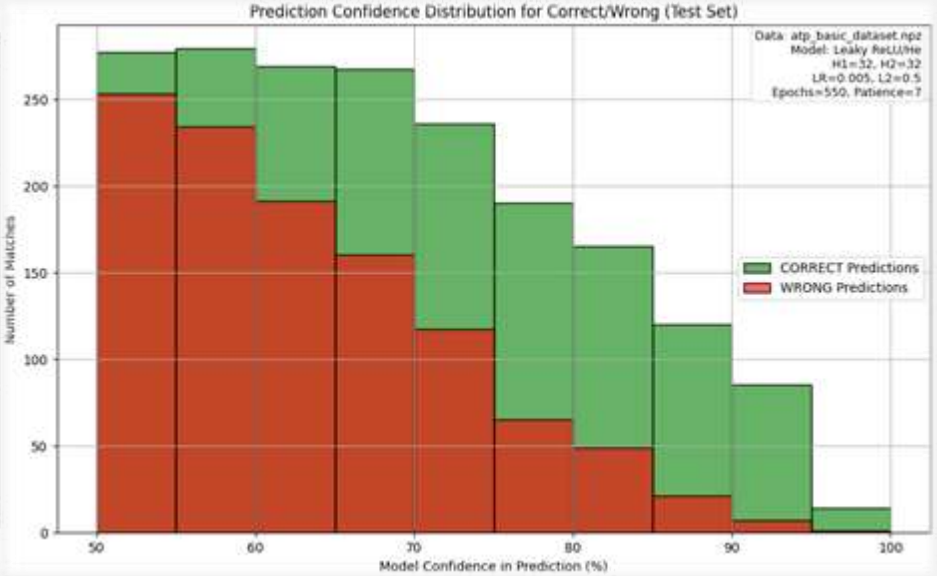
↑ 14 min 30 sec ~

Improving the Model



Improving the Model



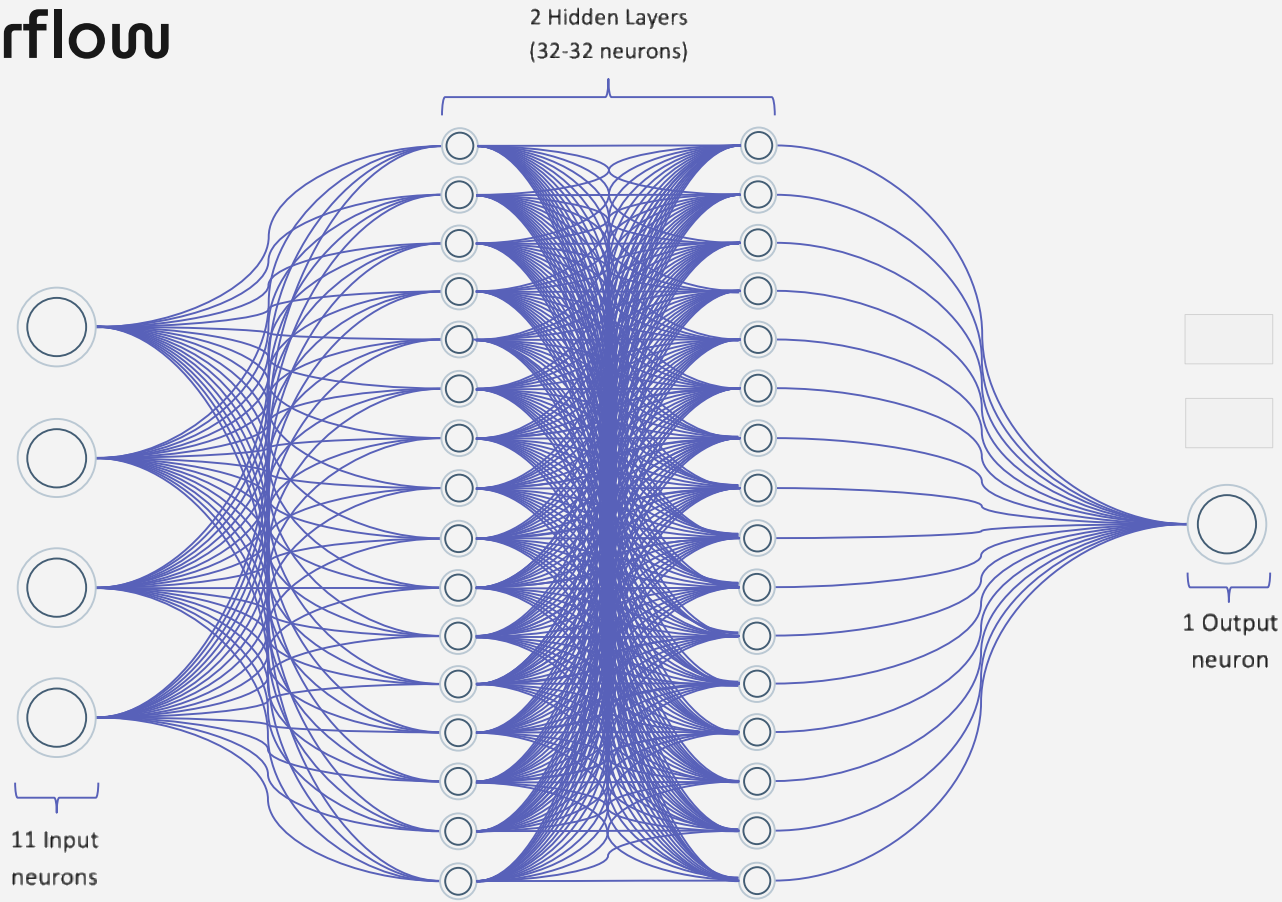


↑ **63.50%**
Test Accuracy

↓ 1,731 parameters

↓ 1 min 30 sec ~

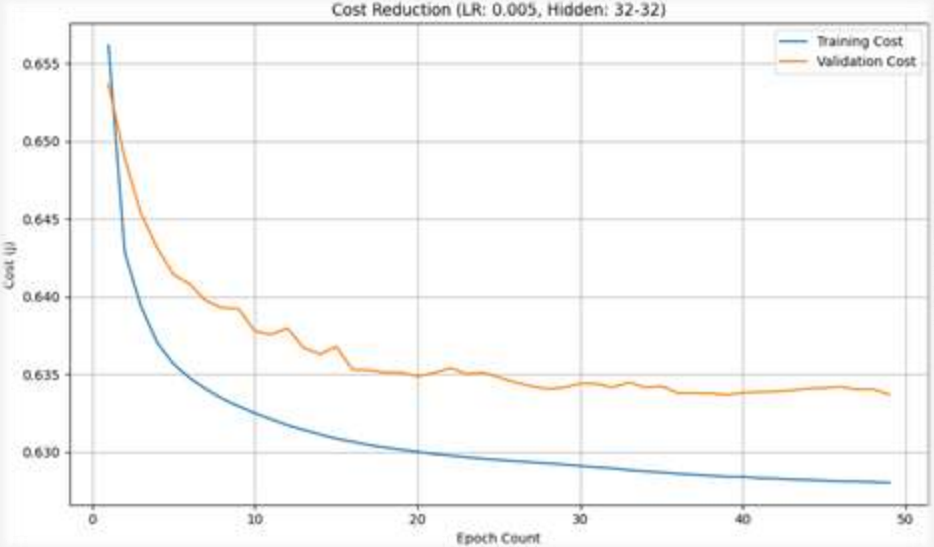
Using Tensorflow

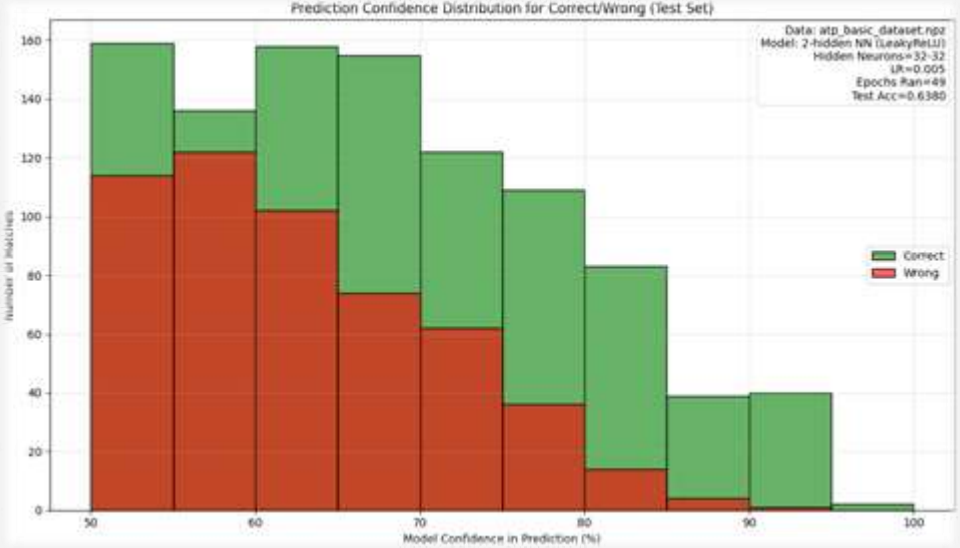


Using Tensorflow

```
def build_model(input_dim,
                hidden1=32,
                hidden2=32,
                l2_lambda=1e-4,
                learning_rate=1e-3):
    he_init = initializers.he_normal()
    l2_reg = regularizers.l2(l2_lambda)

    inputs = layers.Input(shape=(input_dim,), name="input")
    x = layers.Dense(hidden1,
                     kernel_initializer=he_init,
                     kernel_regularizer=l2_reg,
                     use_bias=True,
                     name="dense_hidden_1")(inputs)
    x = layers.LeakyReLU(alpha=0.1)(x)

    x = layers.Dense(hidden2,
                     kernel_initializer=he_init,
                     kernel_regularizer=l2_reg,
                     use_bias=True,
                     name="dense_hidden_2")(x)
    x = layers.LeakyReLU(alpha=0.1)(x)

    outputs = layers.Dense(1,
                           activation='sigmoid',
                           kernel_initializer=he_init,
                           kernel_regularizer=l2_reg,
                           name="output")(x)

    model = models.Model(inputs=inputs, outputs=outputs, name="2hidden_leakyrelu_model")

    opt = tf.keras.optimizers.Adam(learning_rate=learning_rate)
    model.compile(optimizer=opt, loss='binary_crossentropy', metrics=['accuracy'])
    return model
```

Using Tensorflow





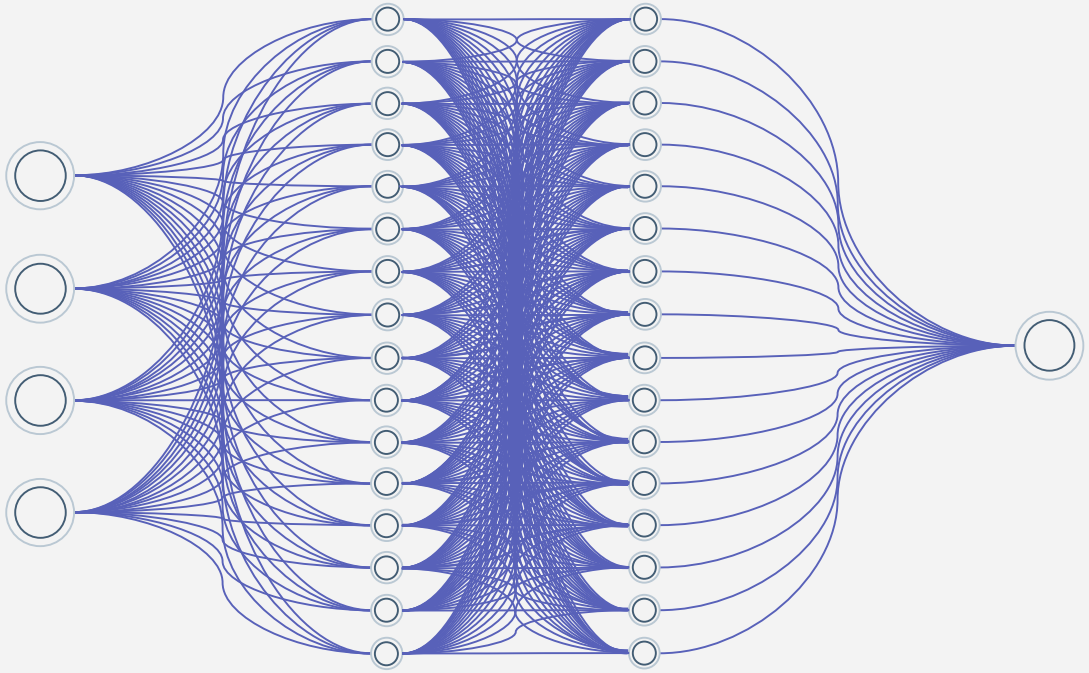
↓ **63.15%**
Test Accuracy

1,731 parameters

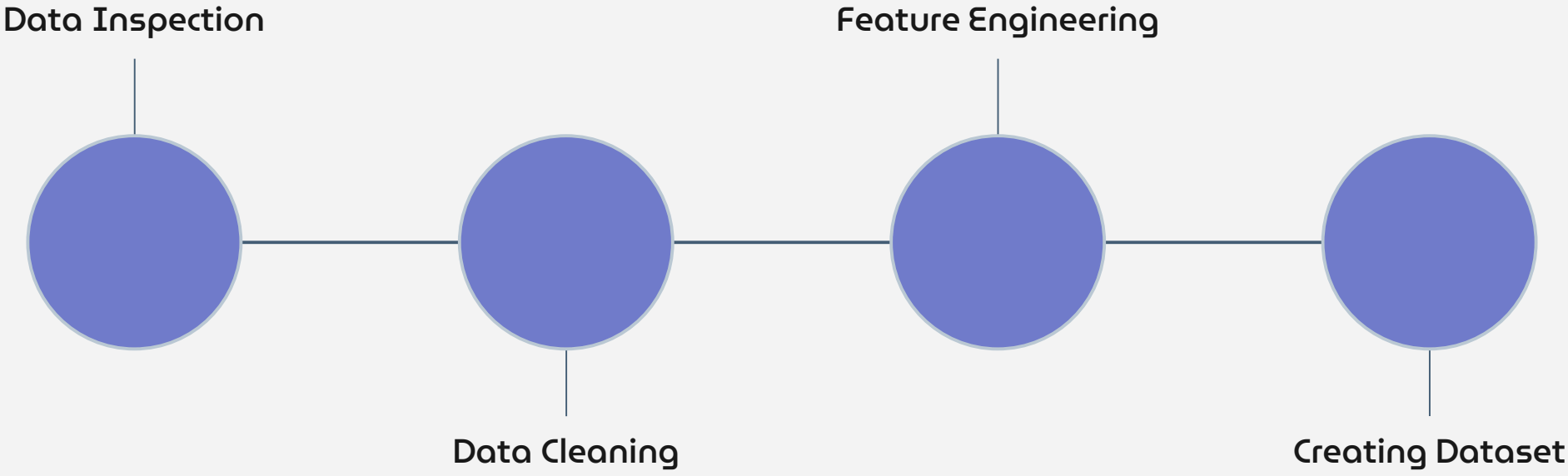
↓ **1 min ~**

Feature Engineering

DATA



Feature Engineering



Data Inspection

Understand the dataset, identify quality issues, and determine feature engineering strategy.

1. Dataset Summary:
- 194,996 matches (1968-2024)
- 56 years of data
- 48 features
- 92.63% data completeness

2. Missing Data Analysis:
- winner_entry: 84.61% missing → WILL NOT BE USED
- loser_entry: 84.61% missing → WILL NOT BE USED
- winner_seed: 58.20% missing → WILL NOT BE USED
- loser_seed: 58.20% missing → WILL NOT BE USED

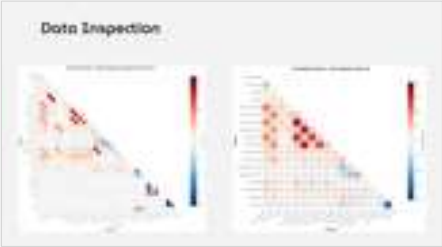
3. Outlier Detection:

a) Rank Outliers:
- Rule: Rank > 2000 → Outlier

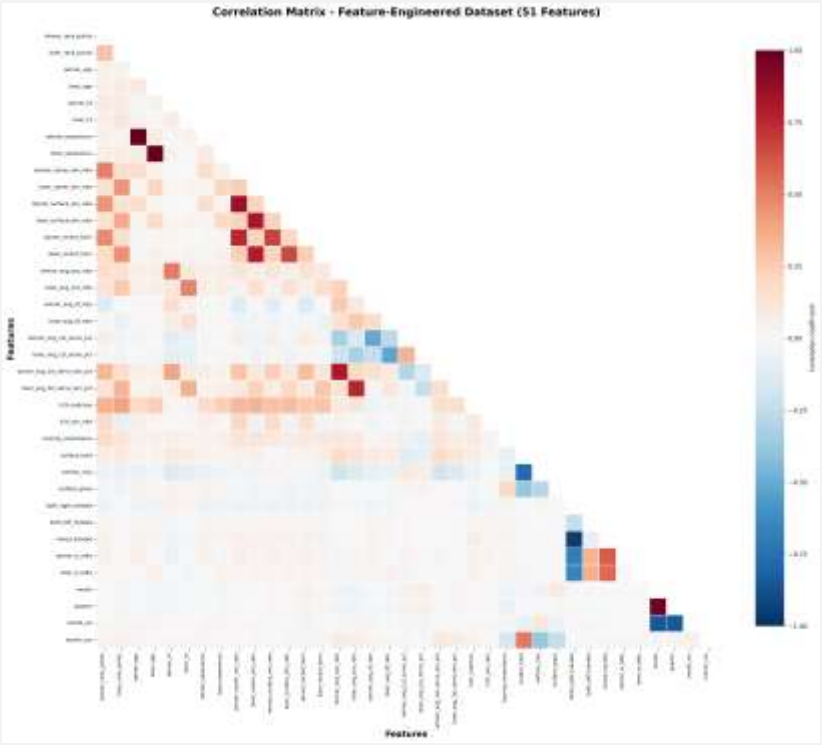
b) Age Outliers:
- Rule: Age < 15 or > 50 → Outlier

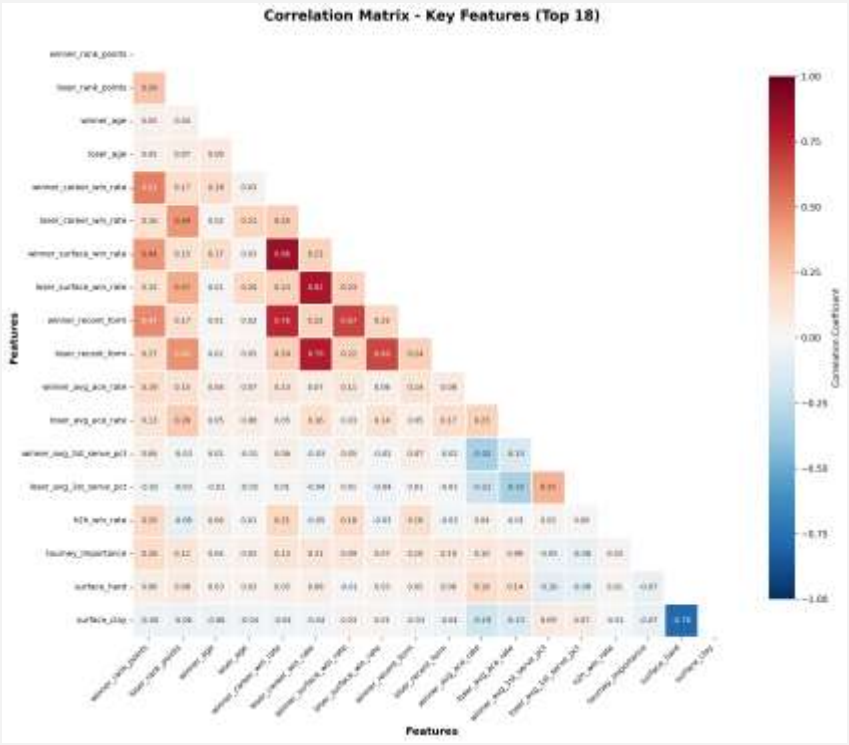
4. Correlation Matrix:

Correlation of features

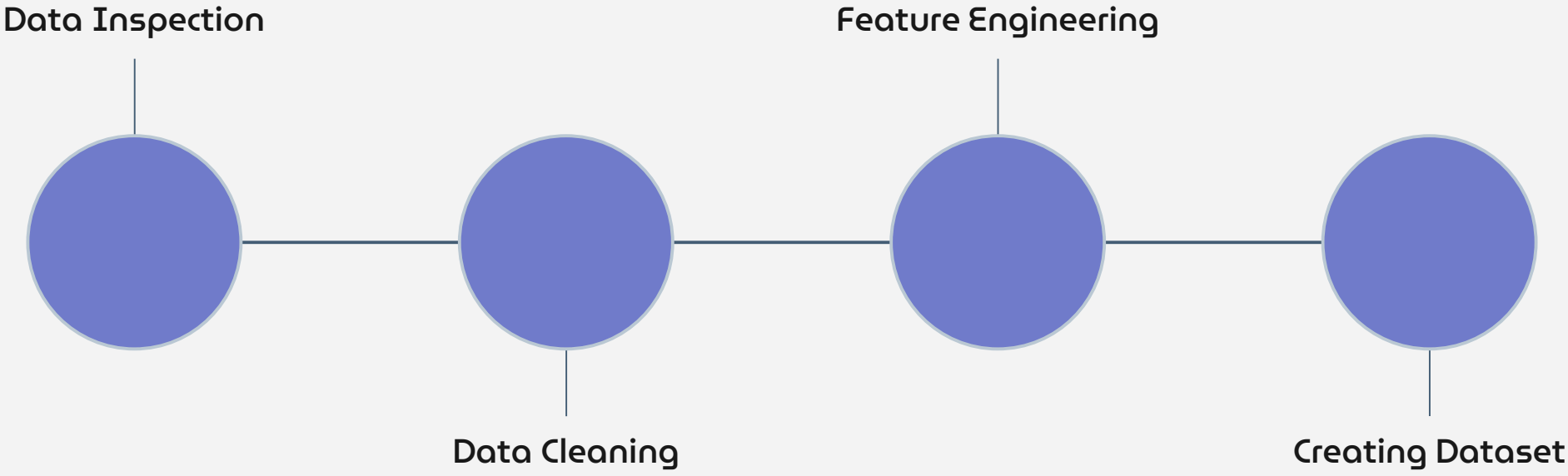


Data Inspection





Feature Engineering



Data Cleaning

Improve data quality, fill missing values, clean outliers, and prepare for model.

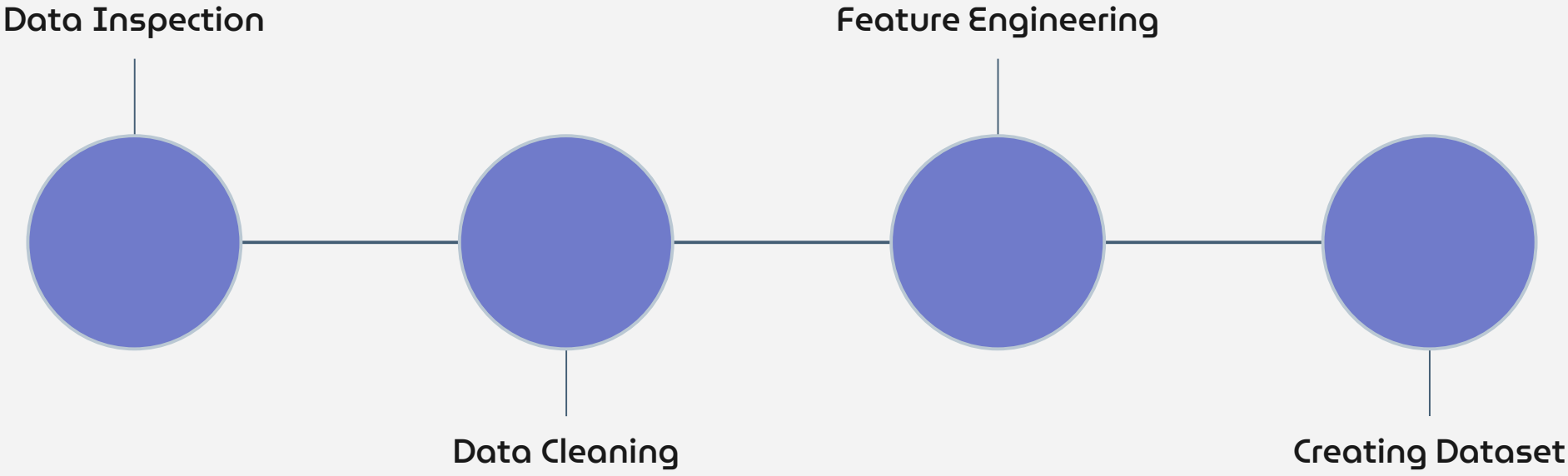
1. Remove High-Missing Columns:
- winner_entry, loser_entry (84.61% missing)
- winner_seed, loser_seed (58.20% missing)
- Reason: Too much missing data, worthless for model

2. Outlier Detection and Cleaning:

a) Rank Outliers:
- Rule: Rank > 2000 → Outlier
- Action: Remove

b) Age Outliers:
- Rule: Age < 15 or > 50 → Outlier
- Action: Remove

Feature Engineering



Feature Engineering

Extract meaningful features from raw data for the model.
CRITICAL: Future information must not be used

1. What is data leakage:
- Using information in training that we wouldn't know when making real predictions.

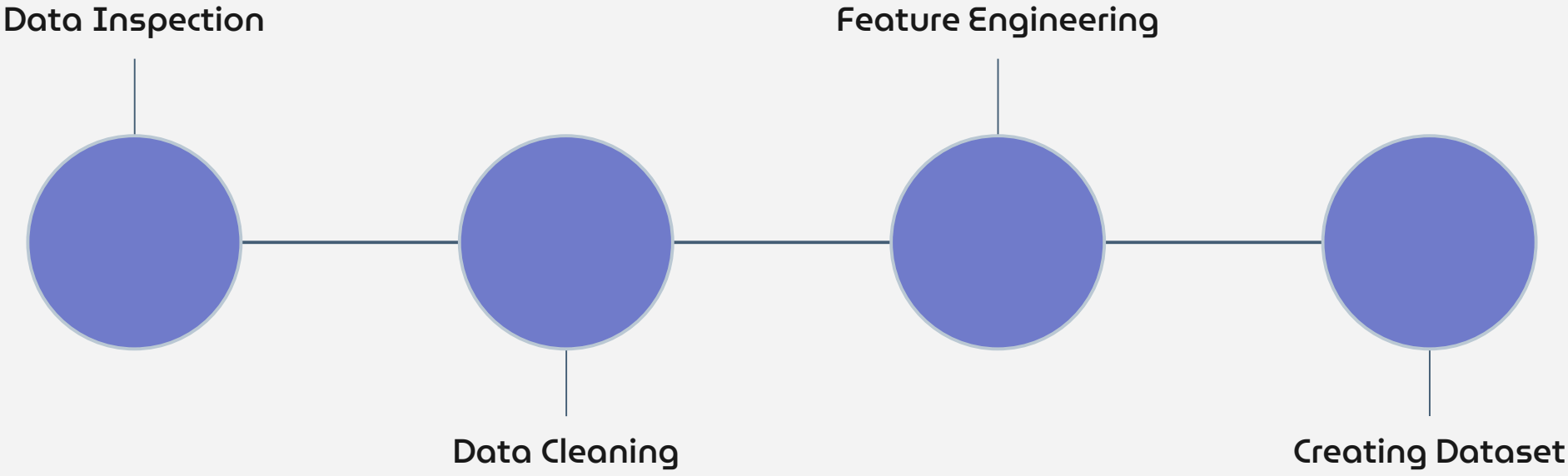
Example (wrong):
- Match: Nadal vs Federer
- Feature: w_ace (winner's ace count)
- Problem: We can't know winner's aces before match starts!

Example (correct):
- Match: Nadal vs Federer
- Feature: nadal_avg_ace_rate (Nadal's ace average in last 20 matches)
- Correct: This information is available before the match!

2. What we did:

Chronological Ordering
Historical Statistics
Historical Service Statistics
Head-to-Head (H2H) Statistics
Tournament Context
Hand Matchup
Data Leakage Cleanup

Feature Engineering



Creating Dataset

Prepare feature-engineered data for the model

1. Data Split:

a) Training Set (1968-2021):
- 186,017 matches (95.4%)
- 54 years of data
- Model will be trained on this data

b) Validation Set (2022-2023):
- 5,903 matches (3.0%)
- 2 years of data
- For hyperparameter tuning
- For early stopping

c) Test Set (2024):
- 3,076 matches (1.6%)
- 1 year of data
- Final performance evaluation
- Model NEVER saw this data!

2. Data Anonymization:

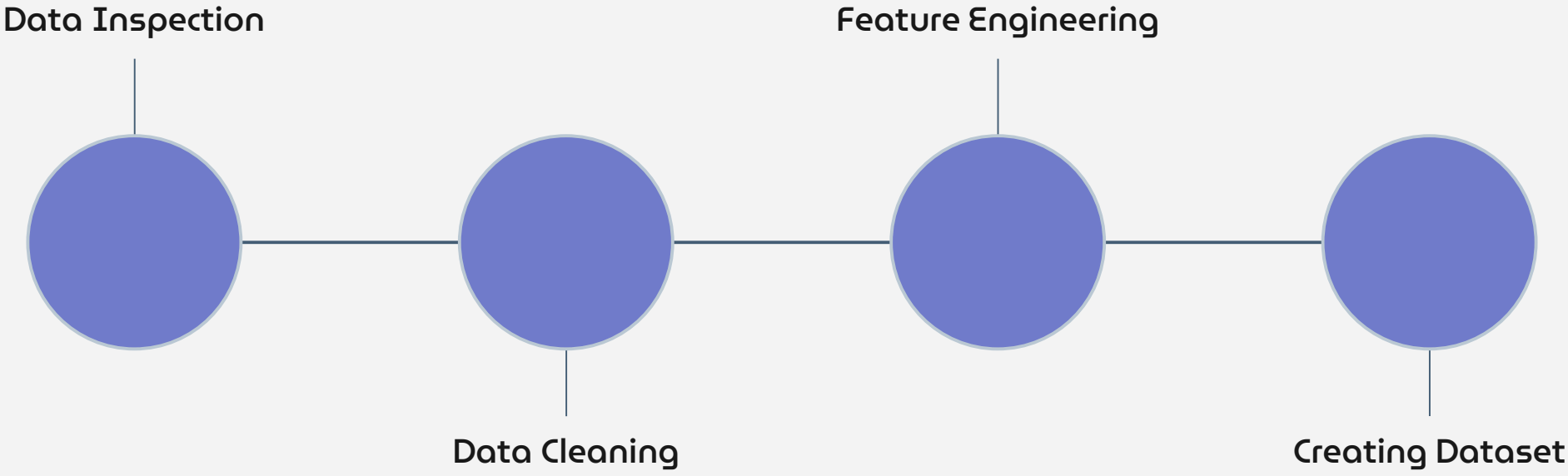
Problem:
- Data: winner_rank, loser_rank
- Model: Could learn "winner always has better rank"
- This is data leakage!

Solution:
- Winner/Loser → Player1/Player2 (random)
- 50% chance: P1=Winner, P2=Loser, Label=1
- 50% chance: P1=Loser, P2=Winner, Label=0

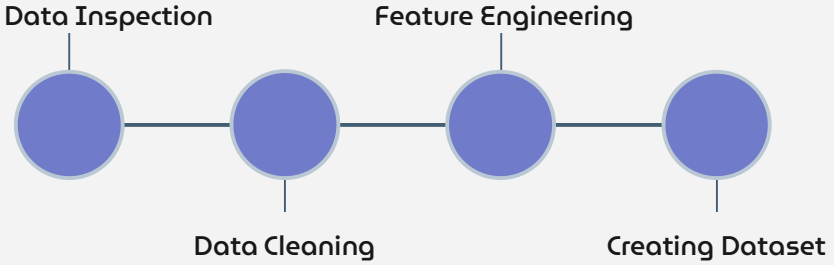
3. Normalization:

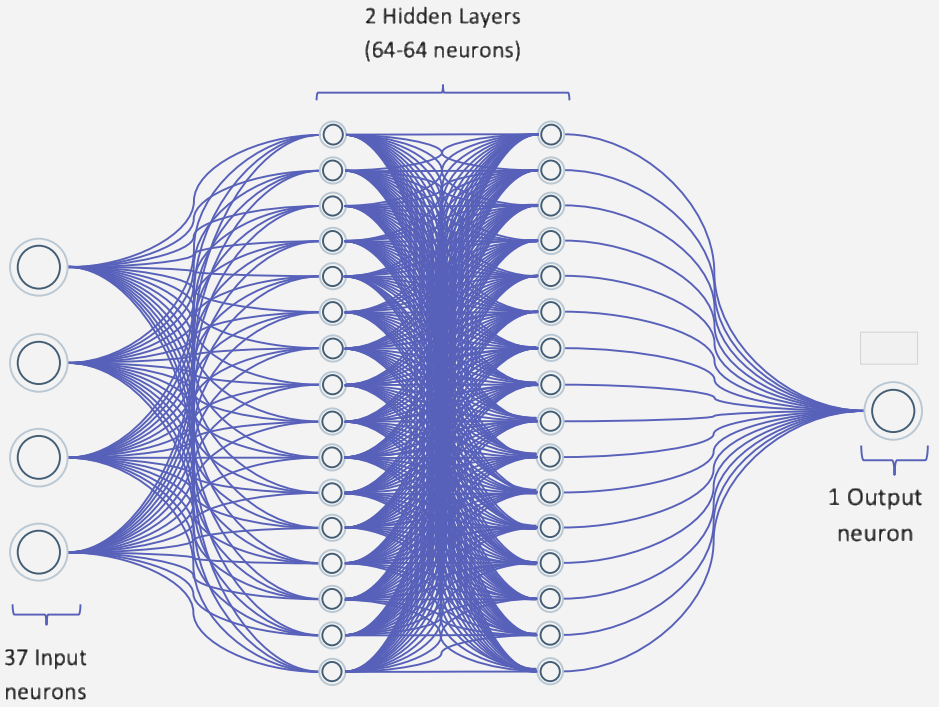
- rank_points: 0-10,000 range
- age: 15-40 range
- ace_rate: 0-0.3 range
- Different scales make model training difficult

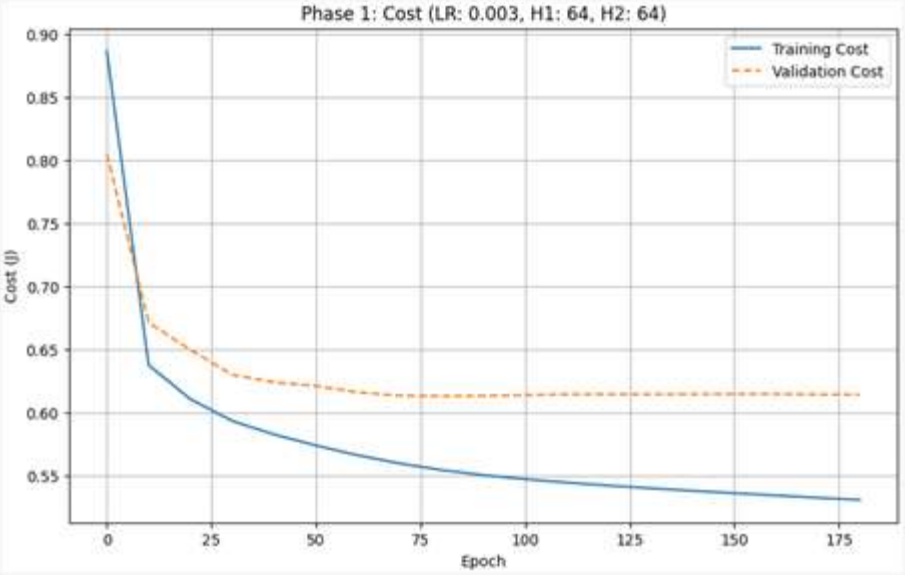
Feature Engineering

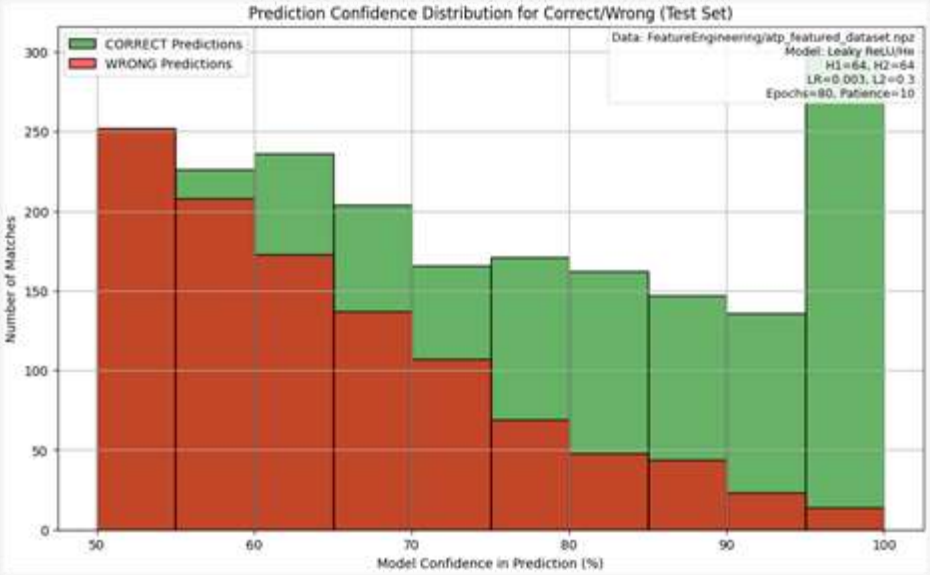


Feature Engineering







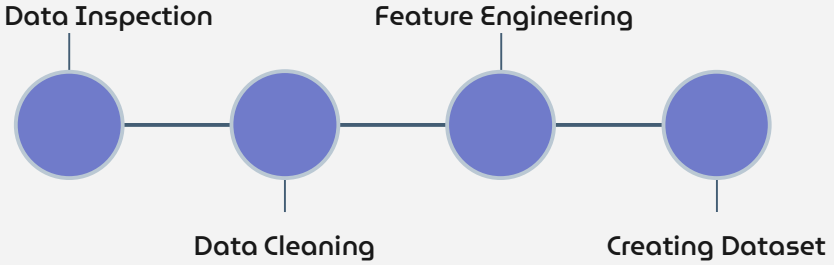


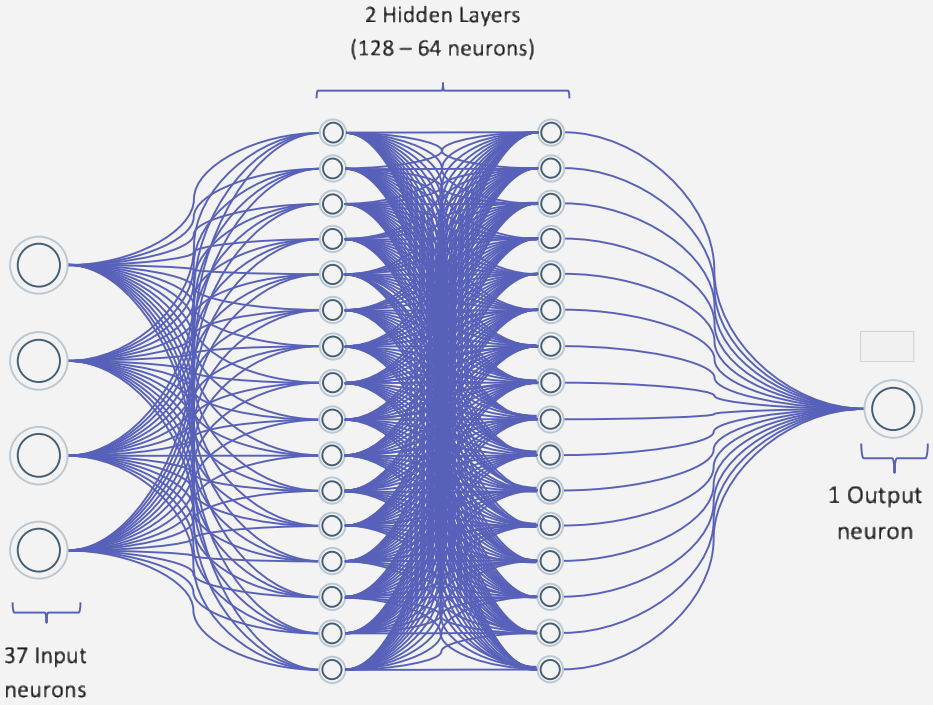
↑ **65.10%**
Test Accuracy

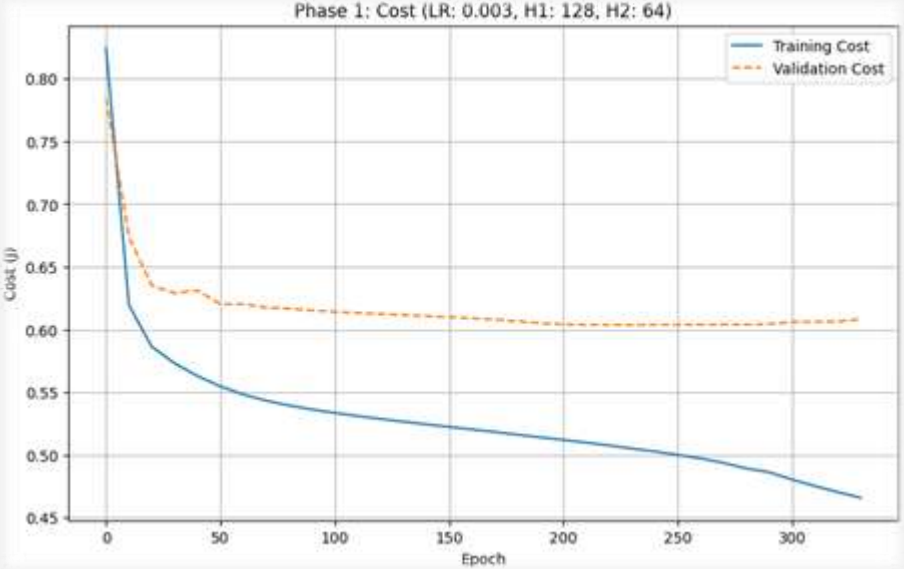
↑ 6,657 parameters

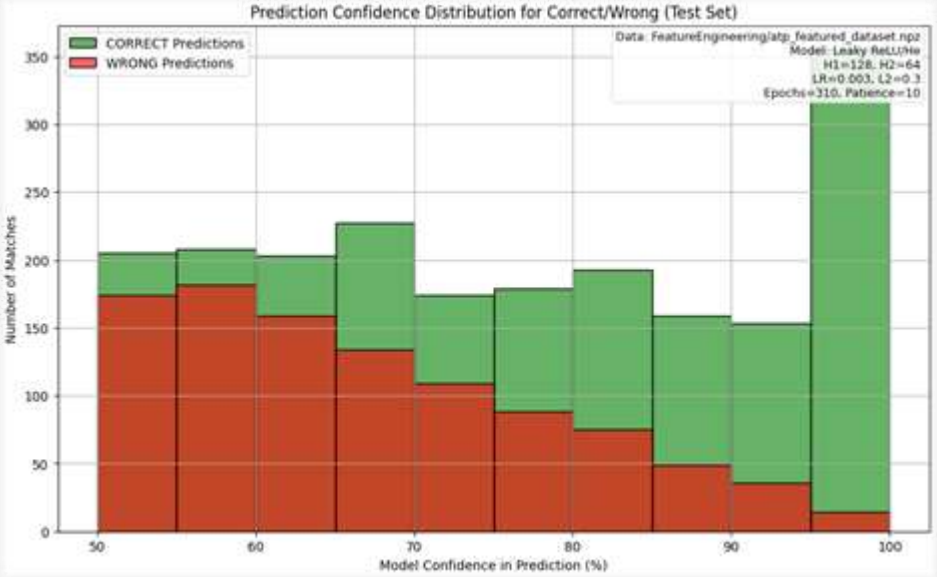
↓ 40 sec ~

Feature Engineering









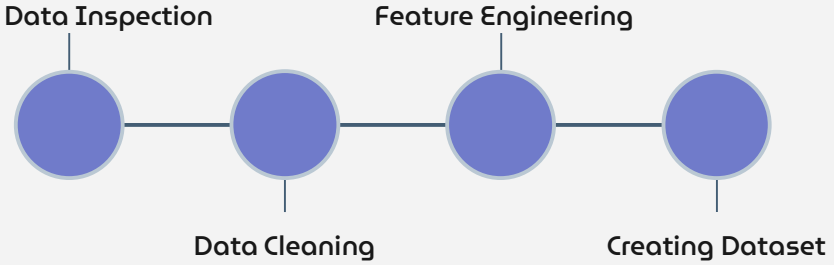
67.35%

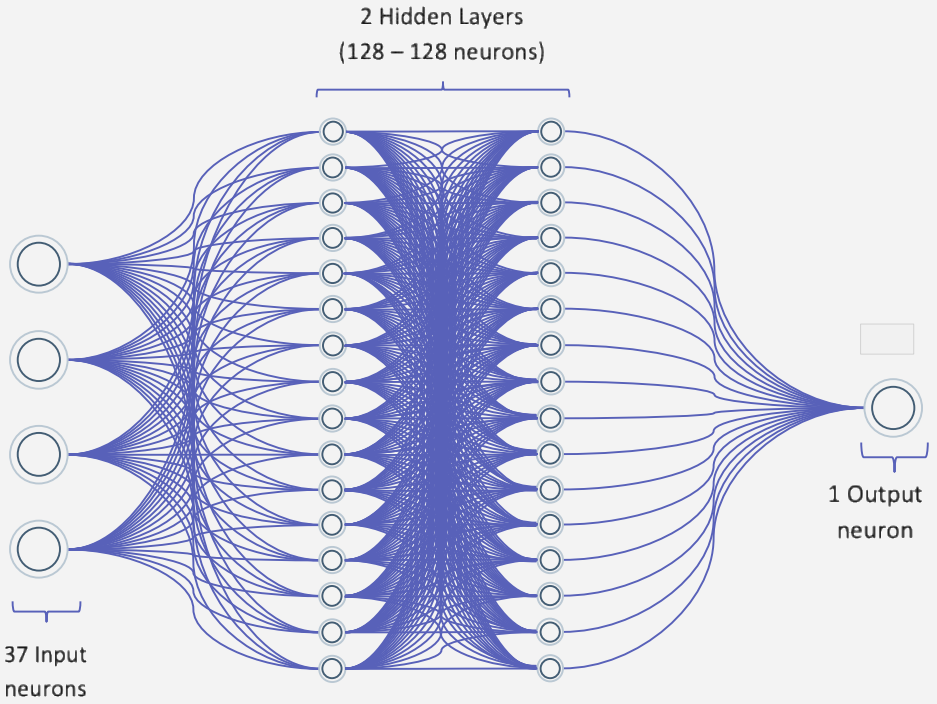
Test Accuracy

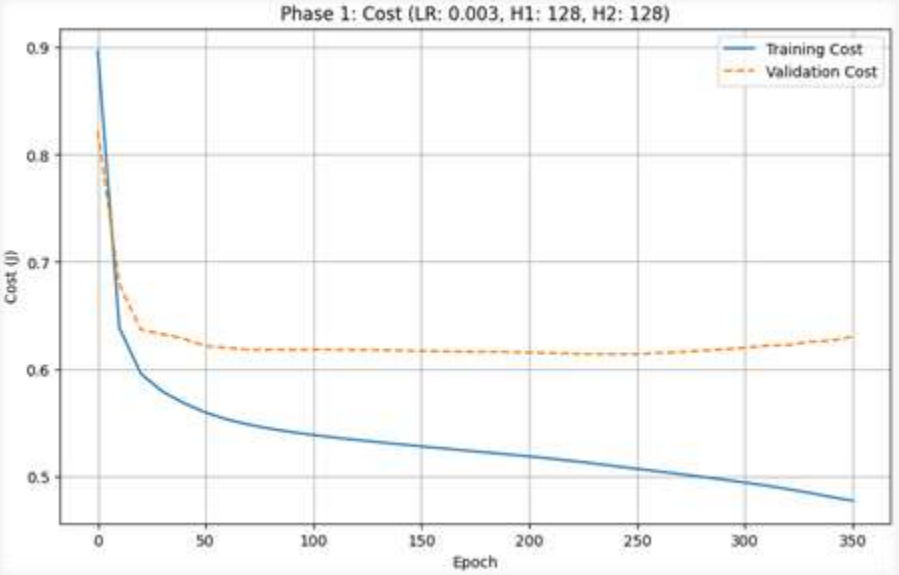
13,185 parameters

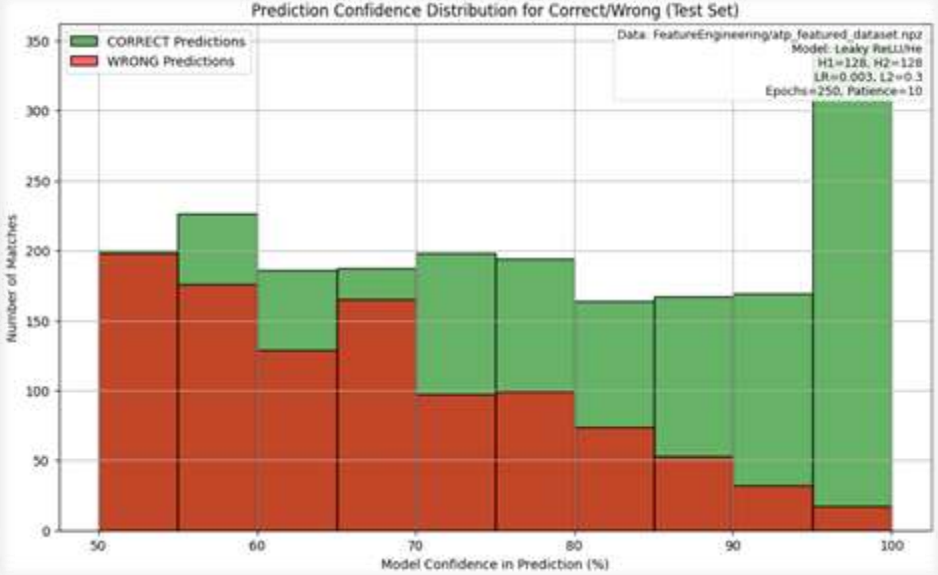
2 min ~

Feature Engineering







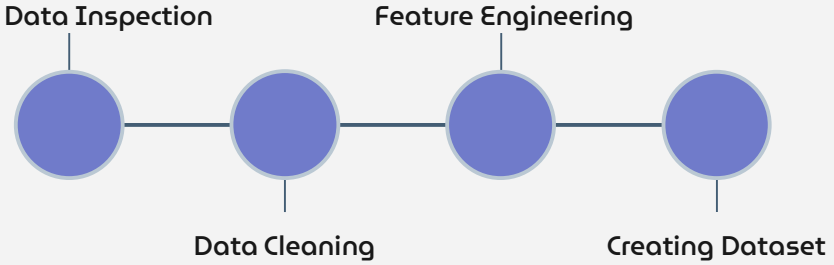


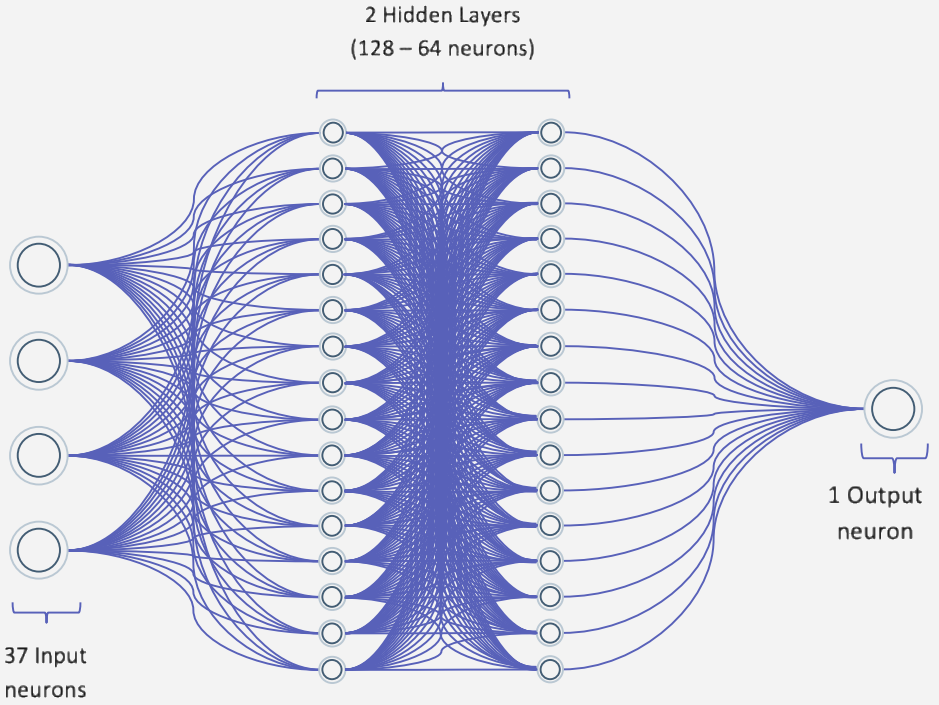
66.20%
Test Accuracy

21,505 parameters

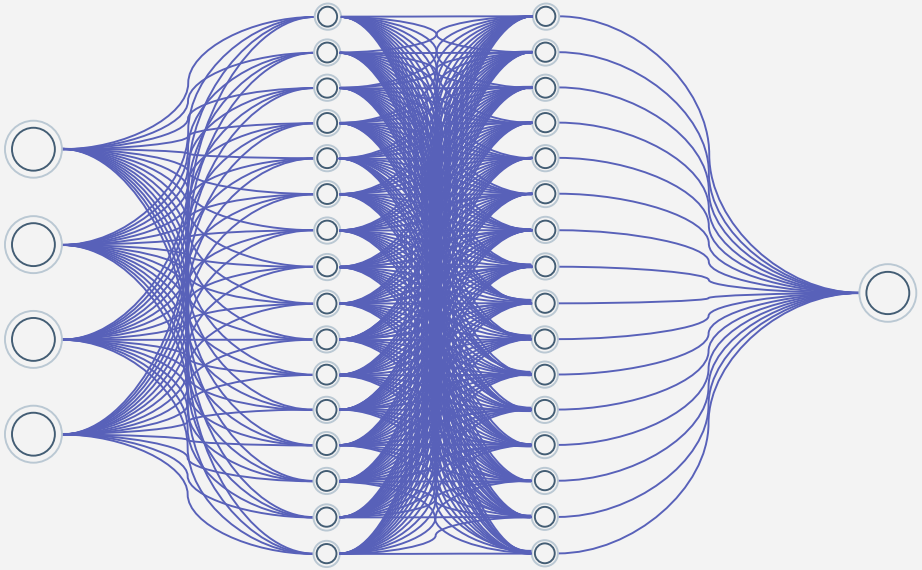
3 min 10 sec ~

Feature Engineering





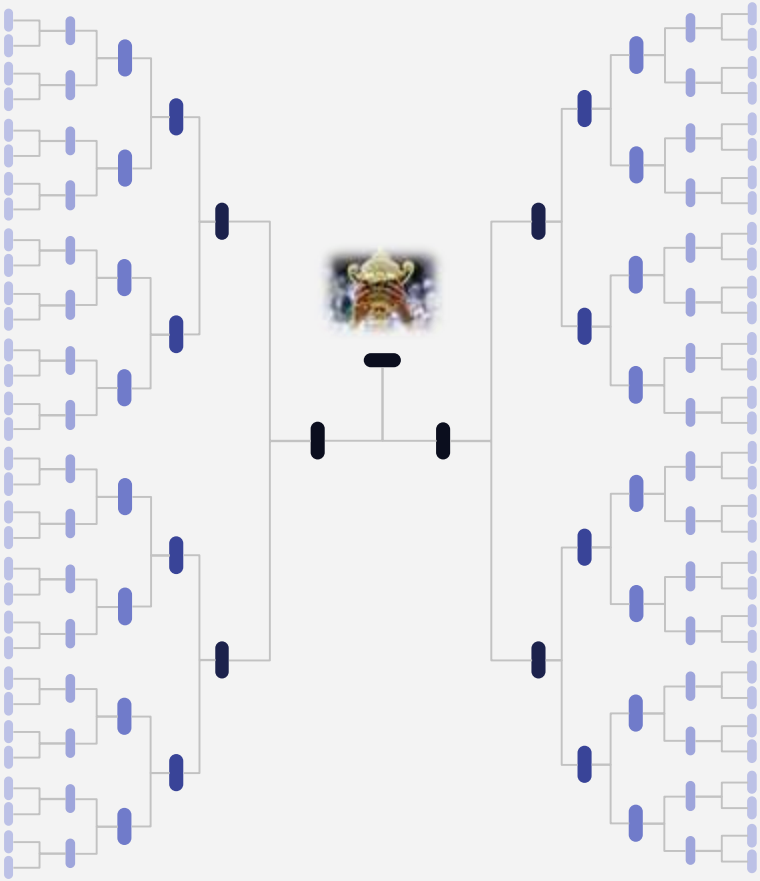
Competition Against IBM

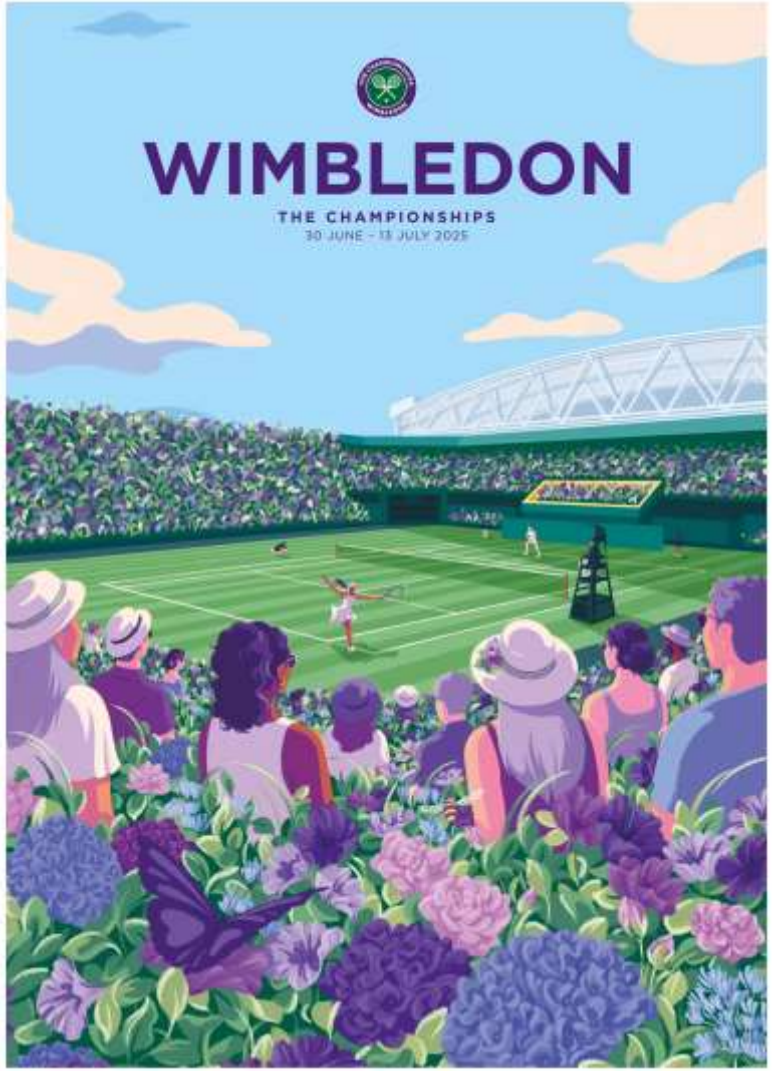


VS

IBM

Wimbledon 2025

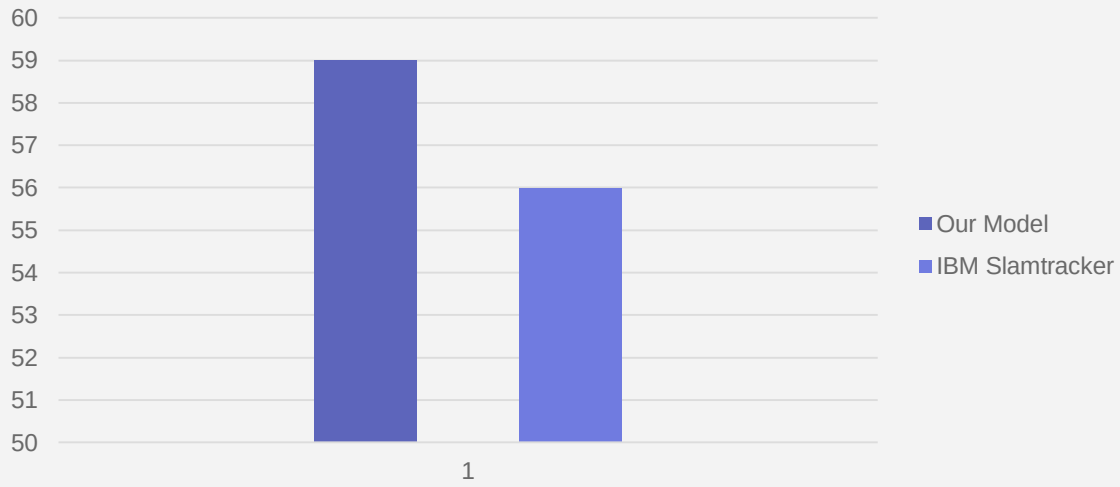




First Round

Player1	Player2	Act	MyP	MyP1%	MyP2%	MyC	✓	IBM	IP1%	IP2%	ICf	✓
Fabio Fognini	Carlos Alcaraz	P2	P2	3.5%	96.5%	96.5%	✓	P2	13%	87%	87%	✓
Zizou Bergs	Lloyd Harris	P2	P1	58.5%	41.5%	58.5%	✗	P1	73%	27%	73%	✗
Adrian Mannarino	C. Connell	P1	P1	94.1%	5.9%	94.1%	✓	P2	42%	58%	58%	✗
Alexandre Muller	N. Djokovic	P2	P1	58.8%	41.2%	58.8%	✗	P2	30%	70%	70%	✓

First Round

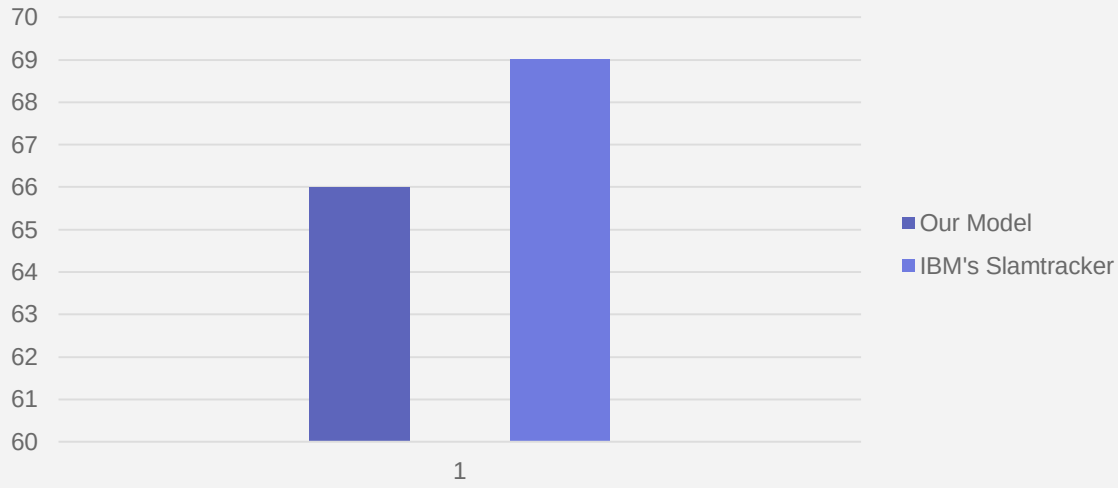


Our Model (38/64) **1** (+2) **0** IBM's Slamtracker (36/64)

Second Round

Player1	Player2	Act	MyP	MyP1%	MyP2%	MyC	✓	IBM	IP1%	IP2%	ICf	✓
Oliver Tarvet	Carlos Alcaraz	P2	P2	1.4%	98.6%	98.6%	✓	P2	6%	94%	94%	✓
Nuno Borges	Billy Harris	P1	P2	41.5%	58.5%	58.5%	✗	P1	67%	33%	67%	✓
Nicolas Jarry	Learner Tien	P1	P1	80%	20%	80%	✓	P2	37%	63%	63%	✗
Jack Draper	Marin Cilic	P2	P1	81%	19%	81%	✗	P2	85%	15%	85%	✗

Second Round

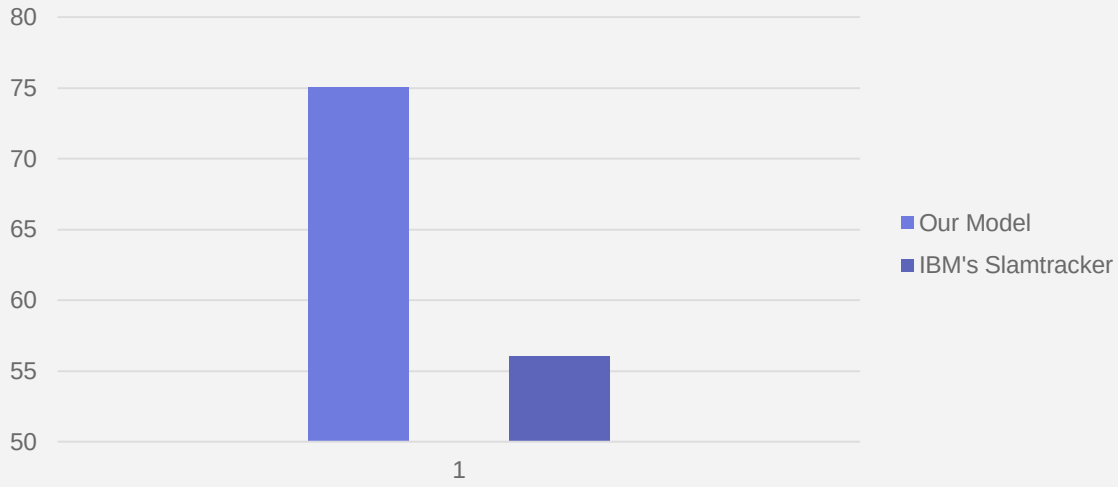


Our Model (21/32) 1 (-1) 1 IBM's Slamtracker (22/32)

Third Round

Player1	Player2	Act	MyP	MyP1%	MyP2%	MyC	✓	IBM	IP1%	IP2%	ICf	✓
Taylor Fritz	D. Fokina	P1	P1	60%	40%	60%	✓	P1	66%	34%	66%	✓
M. Kecmanovic	N. Djokovic	P2	P1	67%	33%	67%	✗	P2	23%	77%	77%	✓
Mattia Bellucci	C. Norrie	P2	P2	13%	87%	87%	✓	P2	55%	45%	55%	✗
B. Nakashima	L. Sonego	P2	P1	65%	35%	65%	✗	P1	51%	49%	51%	✗

Third Round

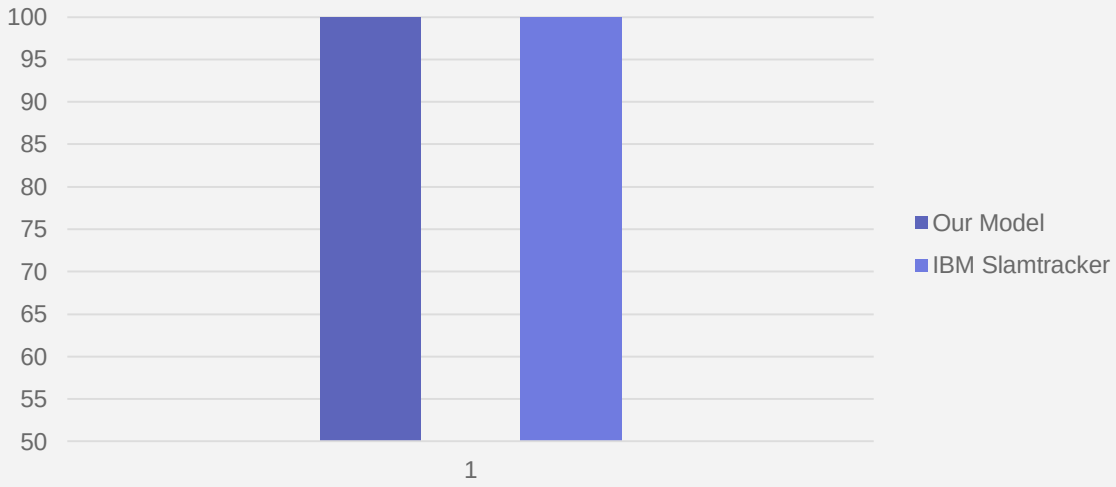


Our Model (12/16) **2** (+3) **1** IBM's Slamtracker (9/16)

Fourth Round

Player1	Player2	Act	MyP	MyP1%	MyP2%	MyC	✓	IBM	IP1%	IP2%	ICf	✓
Andrey Rublev	Carlos Alcaraz	P2	P2	10%	90%	90%	✓	P2	25%	75%	75%	✓
Taylor Fritz	J. Thompson	P1	P1	80%	20%	80%	✓	P1	77%	23%	77%	✓

Fourth Round

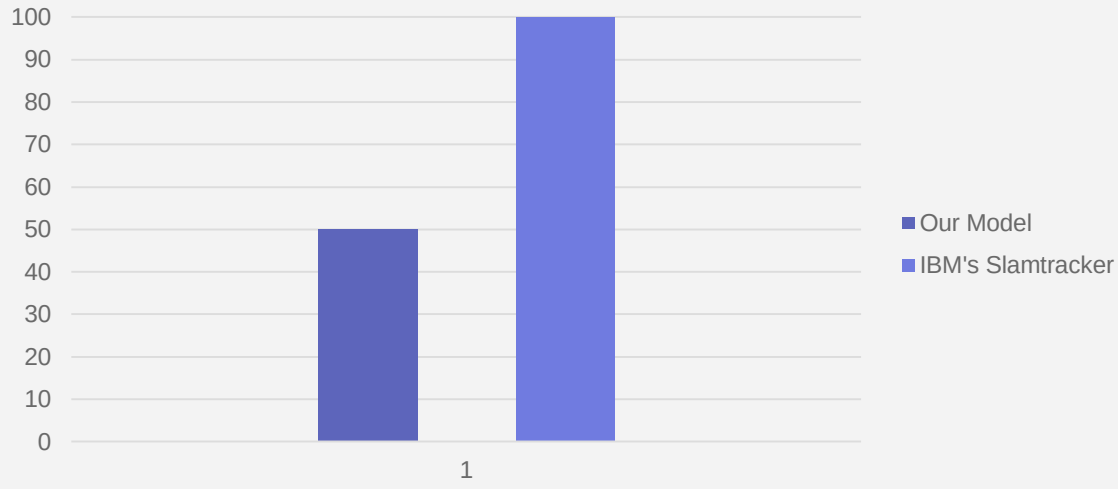


Our Model (8/8) **2** (0) **1** IBM's Slamtracker (8/8)

Quarter Finals

Player1	Player2	Act	MyP	MyP1%	MyP2%	MyC	✓	IBM	IP1%	IP2%	ICf	✓
Cameron Norrie	Carlos Alcaraz	P2	P2	3%	97%	97%	✓	P2	13%	87%	87%	✓
Taylor Fritz	K. Khachanov	P1	P2	28%	72%	72%	✗	P1	67%	33%	67%	✓
Flavio Cobolli	N. Djokovic	P2	P1	67%	33%	67%	✗	P2	26%	74%	74%	✓
Jannik Sinner	Ben Shelton	P1	P1	100%	0%	100%	✓	P1	80%	20%	80%	✓

Quarter Finals



Our Model
(2/4)

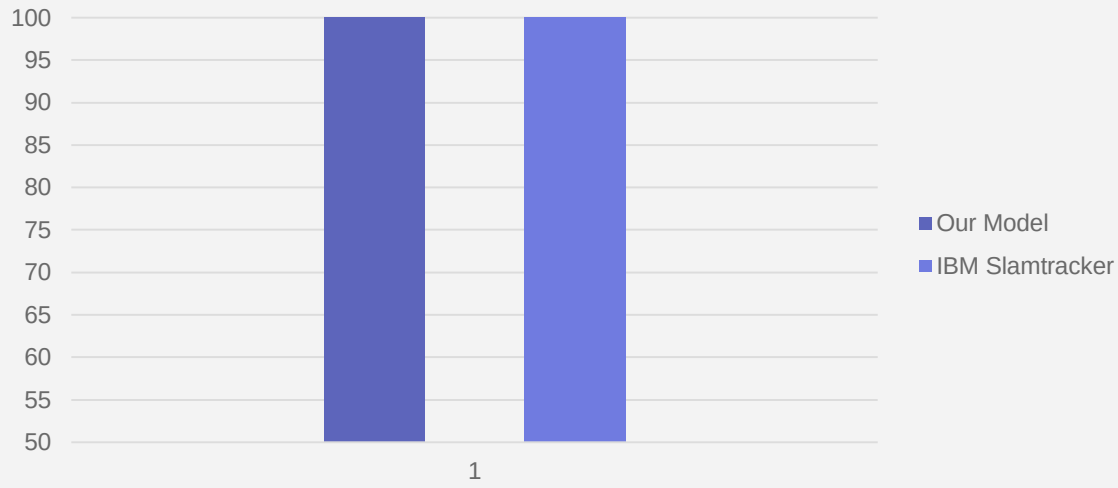
2 (-2) 2

IBM's Slamtracker
(4/4)

Semi Finals

Player1	Player2	Act	MyP	MyP1%	MyP2%	MyC	✓	IBM	IP1%	IP2%	ICf	✓
Taylor Fritz	Carlos Alcaraz	P2	P2	35%	65%	65%	✓	P2	40%	60%	60%	✓
Jannik Sinner	N. Djokovic	P1	P1	99%	1%	99%	✓	P1	56%	44%	56%	✓

Semi Finals

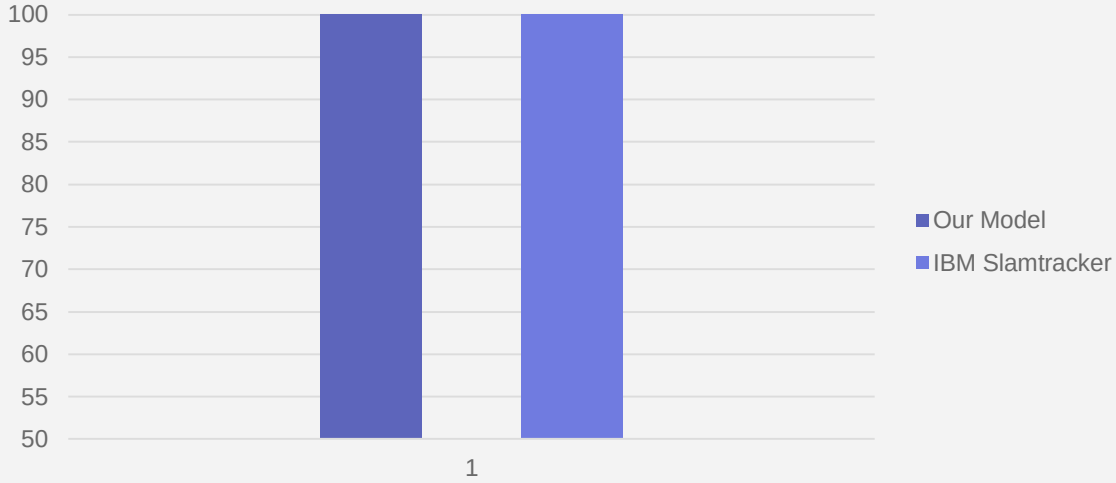


Our Model (2/2) **2** (0) **2** IBM's Slamtracker (2/2)

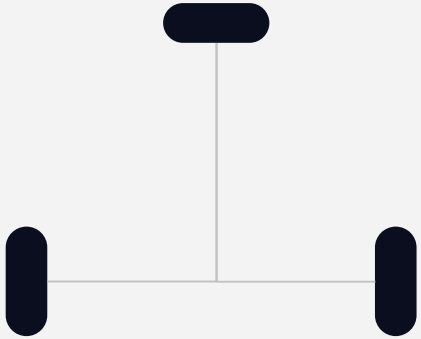
Final

Player1	Player2	Act	MyP	MyP1%	MyP2%	MyC	✓	IBM	IP1%	IP2%	ICf	✓
Jannik Sinner	Carlos Alcaraz	P1	P1	99%	1%	99%	✓	P1	51%	49%	51%	✓

Final



Jannik Sinner



Our Model
(1/1)
Total (84/127)

2 (0) 2

IBM's Slamtracker
(1/1)
Total (82/127)

Lessons Learned & Future Work

01

The Calibration Problem

02

Probability Calibration

03

The Outlier (Djokovic Paradox)

04

Data Freshness & Coverage

The Calibration Problem

So, why is my model so sure of itself? In the literature, this is known as an 'Uncalibrated Probability' problem.

Concept: Uncalibrated Probabilities

Cause

Sigmoid Function: Pushes values to extremes (0 or 1).

Cross-Entropy Loss: Penalizes uncertainty, forcing the model to be "sure."

The Diagnosis: The model is good at ranking (Accuracy), but bad at estimating the true probability (Calibration).

Term: Overconfidence.

Probability Calibration

Problem: The model is Overconfident (Uncalibrated).

Prediction: 95% vs Reality: 60%.

Solution: Temperature Scaling.Apply a temperature parameter (T) to the

Softmax function:

$$\text{softmax}\left(\frac{\text{logits}}{T}\right)$$

goal: "Soften" the probabilities to match real-world uncertainty (like IBM).

The Outlier (Djokovic Paradox)

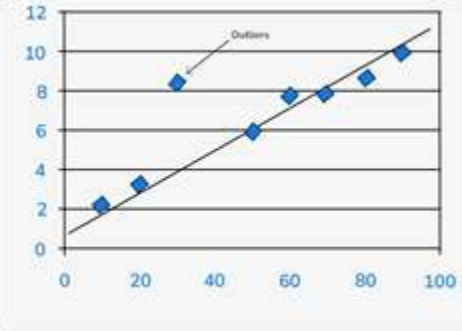
I realized my model struggled specifically with Novak Djokovic, with an error rate 50%. IBM, however, predicted these matches correctly.

The Missing Link: Latent Variables (Hidden Factors).

Mental Toughness

Clutch Points

The Diagnosis: Eventough he is old still he can play top-level, Novak Djokovic has won 3 grand slams in 2023 at age 36.



Data Freshness & Coverage

Beyond specific player psychology, I identified a systematic issue regarding time. My model predicts the 2025 tournament using data that ends in 2024.

Scenario: Validated on '22-'23 -> Tested on '24 -> Predicting 2025.

The gap: Predicting a Jan 2025 match using stats frozen in late 2024.

Impact: Misses immediate "Current Form" and "Momentum."

Example, a player improved significantly during the off-season or pre-tournament warm-ups, but the model is blind to this.

Solution: Implement Rolling Window Updates (Retrain model weekly with latest match results).

Data Freshness & Coverage

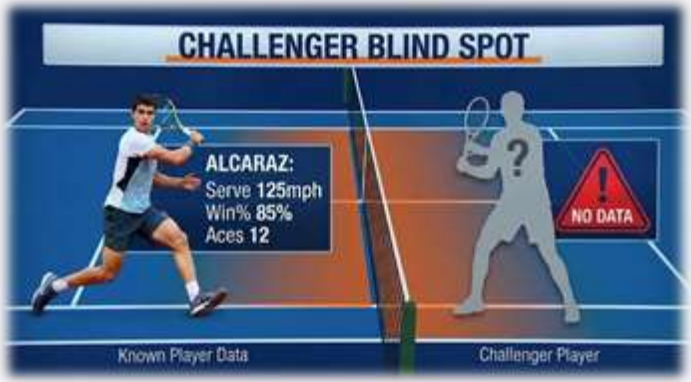
The "Challenger" Blind Spot (Early Rounds)

Observation: Lower accuracy in Round 1 & 2 compared to Finals.

Cause: Qualifiers coming from Challenger Tournaments.

Issue: The dataset focuses on major ATP events. Newcomers appear as "Zeros" or missing data.

Solution: Integrate Challenger/ITF Tour Data to capture the stats of rising stars before they enter Grand Slams.



Recap

The Foundation

Built a Neural Network from scratch using NumPy

The Evolution

Scaled from a basic model to a Deeper 2-Layer Network with modern optimizations (Adam, He, LReLU).

The Validation

Verified accuracy against industry standards (TensorFlow) and real-world benchmarks (IBM).

The Takeaway

Algorithms are powerful, but Data is King.

Understanding the "Why" (Math) is more important than just coding the "How".

Thanks!

Do you have any questions?